CS 학부 과제 안내 · HW #3 · 학생용 가이드

데이터통신 프로그래밍 과제

Fault-Tolerant 협업 Worker Ring

반복되는 Crash · Restart · Rejoin 속에서도 정확히 끝나는 분산 작업

Thread Pool | Mutex / Lock Ordering | Condition Variable | Heartbeat | Checkpoint | Takeover | Epoch | Rejoin

DUE

2026.10.26 (월) 23:59

Assigned 2026.10.13 (화) 10:30

제출 방식

G조이름HW3.zip (조별 1인 제출)

ex) G2HW3.zip · 모든 소스 + 결과(로그) 파일

AGENDA

오늘 다룰 내용

01

과제 한눈에

개요 · HW#1·2와 비교

02

쉬운 비유 · 핵심 개념

협업 번역 팀으로 이해

03

시스템 구조 · 배포

Coordinator · Worker · Launcher

04

계산 작업

Job · Task · step · 소수 세기

05

장애 내성 메커니즘

HB · CKPT · Takeover · Epoch · Rejoin

06

Task 흐름 따라가기

배정 → 계산 → 반납 → 인계

07

동기화 · 프로토콜

Mutex · Lock · CV · 메시지 규칙

08

시나리오 · 평가 · 제출

S1~S10 · 로그 · 지표 · FAQ

OVERVIEW

과제 한눈에 보기

이번 과제는 '장애 내성(Fault Tolerance)' 문제를 다룹니다

Worker 10개가 링(Ring)을 이루어 1부터 1억까지의 소수를 나누어 세는 동안, 무작위로 고른 Worker들이 몇 번이고 죽었다 살아나도(동시에 최대 2개) 최종 답 5,761,455가 정확히 한 번씩만 더해져 나오게 만듭니다.

- **무엇을** 원격 Coordinator 1개 + 로컬 Worker 10개 + Chaos Launcher 1개
- **어떻게** 1억까지의 소수 세기를 Task 1,000개로 나눠 Worker가 요청(REQ)해서 계산
- **장애** Launcher가 무작위 Worker를 crash → 얼마 뒤 restart → rejoin (반복)
- **핵심** crash가 몇 번 나도 모든 Task가 정확히 1번 반영 (이중 Commit 0건)
- **구현** 개념은 실제 장애, 구현은 가상(virtual) crash — 연결 유지, 상태만 지움

HW#2 동시성 → HW#3 동시성 + 장애 내성

10 Worker (고정 Ring)	**1,000** Task (각 수 100,000개)
≤ 2 동시에 crash된 Worker	**5,761,455** 정답 (1~10^8의 소수 개수)

COMPARISON

HW#1 → HW#2 → HW#3, 무엇이 이어지고 무엇이 새로운가

항목	HW#1 (분산)	HW#2 (동시성)	HW#3 (장애 내성)
구조	Master / Worker 분산	단일 서버 + Thread Pool	원격 Coordinator + 로컬 Worker 10개 Ring
풀어야 할 문제	일을 나눠 맡기기	스레드 간 자원 경합	Worker가 반복해서 죽었다 살아남
핵심 도구	Socket, 작업 분배	Mutex, Lock Ordering, CV	Heartbeat, Checkpoint, Takeover, Epoch, Rejoin
성공 기준	작업 완료	이중예약 0 · Deadlock 0	1,000 Task DONE · 이중 Commit 0 · 합계 5,761,455
다시 쓰는 것	—	—	HW#1 분배 + HW#2 Thread Pool·Lock·CV 그대로

한 줄 요약: HW#2의 “이중예약”이 HW#3에서는 “이중 Commit”으로 다시 등장합니다. 같은 Task의 결과가 두 번 더해지면 안 됩니다.

EASY ANALOGY

온라인 협업 번역 팀으로 이해하기

출판사 PM 한 명이 번역가 10 명과 책 1,000 쪽을 나누어 번역한다고 생각해 봅시다. 노트북은 가끔 갑자기 꺼집니다.



Coordinator

PM (프로젝트 관리자)

쪽(Task)을 나눠 주고 완성본을 모읍니다. 누가 꺼졌는지 판단합니다.



Worker 10개

번역가 10명

받은 쪽을 문단(step) 단위로 번역합니다. 옆자리와 고리로 앉아 있습니다.



Heartbeat (HB)

"온라인 표시" 1초 갱신

5초 이상 갱신이 없으면 PM은 오프라인(DEAD)으로 간주합니다.



Checkpoint (CKPT)

옆자리 공유 폴더 자동 백업

문단을 끝낼 때마다 옆 사람(Buddy) 폴더에 진행 상황을 저장합니다.



Takeover

옆 사람에게 이어 맡김

꺼진 사람의 쪽을 다음 사람에게, 백업 지점부터 이어서 맡깁니다.



Epoch

배정 번호(버전)

담당자가 바뀔 때마다 번호를 올려, 옛 번호로 올라온 원고는 반려합니다.

다시 켜진 번역가는 저장 안 된 작업을 잃었으므로 PM에게 복귀를 알리고(Rejoin) 새 쪽을 받습니다. 옆자리 두 명이 동시에 꺼져 있으면 그다음 사람에게 넘깁니다(다음 ALIVE 규칙).

CORE CONCEPTS

핵심 개념 8가지

Heartbeat

Worker가 1초마다 보내는 “살아 있음” 신호

SUSPECT / DEAD

HB가 3초 없으면 의심, 5초 없으면 죽은 것으로 판정

Checkpoint

step마다 진행 상황(누적값)을 Buddy에게 백업

Takeover

죽은 Worker의 Task를 링의 다음 ALIVE Worker가 인계

Epoch

Task 담당자가 바뀔 때마다 +1 되는 번호 (Fencing)

Commit

유효한 RESULT를 최종 확정 → 합계에 한 번만 반영

Rejoin

재시작한 Worker가 재합류를 요청 → 새 Task를 받음

Virtual Crash

연결은 유지한 채 상태를 지우고 침묵하는 가상 장애

성공 기준 **1,000 Task 모두 DONE · 모든 commit_count = 1 · 합계 5,761,455 · crash마다 감지 → 인계 → 재합류 → 새 Task가 로그로 확인**

MODEL

일반 모델과 가상(virtual) 구현

개념은 실제 장애로 설명하지만, 구현은 가상 crash로 합니다



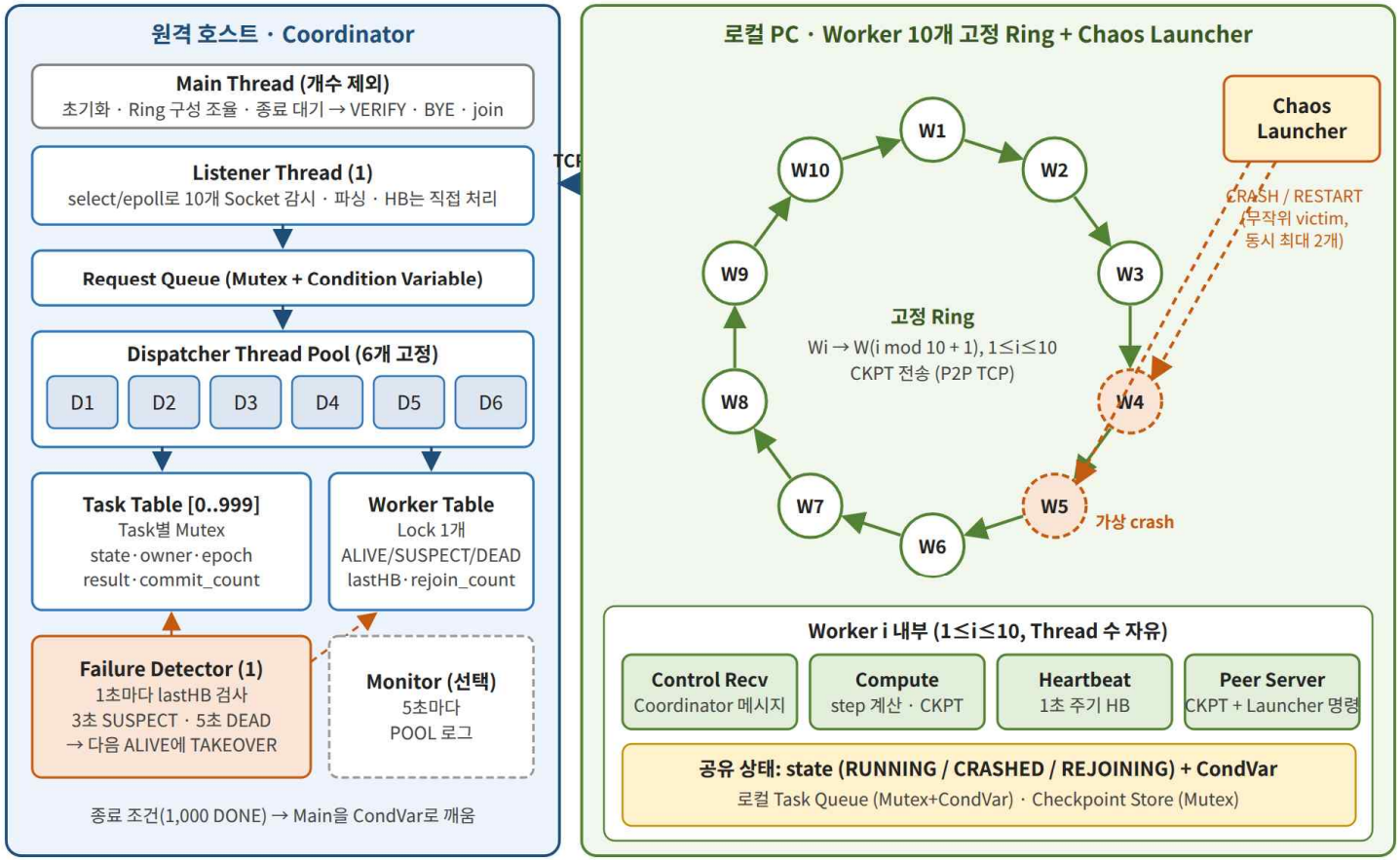
항목	실제 시스템 (개념)	이 과제의 구현 (virtual)
crash 원인	전원 차단, 프로세스 종료	Chaos Launcher가 CRASH 명령을 통보
메모리 상태	사라짐	Worker가 로컬 Queue · 진행 중 Task · Checkpoint Store를 직접 모두 지움
통신	아무것도 보내지 못함	HB·REQ·RESULT·CKPT 모두 보내지 않음. 받은 메시지는 읽어서 버림
TCP 연결	끊어짐(EOF)	그대로 유지 (close 금지) → EOF가 없으므로 HB로만 감지
restart	새 프로세스, 재접속	RESTART 통보 → REJOINING → 같은 연결로 REJOIN

가상이라도 "진짜처럼" 해야 할 3가지

1. crash 동안 정말로 아무것도 보내지 않는다 (HB 하나라도 새면 감지 실패)
2. crash 순간 모든 작업 상태를 버린다 (restart 후 옛 Task를 이어 계산 금지)
3. Coordinator는 Launcher와 통신하지 않고 HB만 보고 판단한다

ARCHITECTURE

전체 시스템 구조 (System Architecture)



Coordinator ×1

원격 호스트 (AWS·GCP 등)

Worker 등록 · Ring 구성 · Task 분배와 합산 · HB로 장애 감지 · 인계 · 재합류 승인

Worker ×10

로컬 PC, 독립 프로세스 10개

Task를 step 단위로 계산 · step마다 CKPT를 Buddy에게 · crash 통보 시 침묵

Chaos Launcher ×1

로컬 PC (Worker와 같은 PC)

Worker 10개 실행 · 무작위 victim에게 CRASH / RESTART 통보 · 동시 crash ≤ 2

Buddy(i) = i mod 10 + 1
(W10의 Buddy는 W1)

DEPLOYMENT

배포 규칙과 연결 3종류 — 모두 TCP

원격 VM (클라우드)

Coordinator 1개
별도의 물리적 원격 호스트 (AWS, GCP 등)
보안 그룹에서 포트 허용

로컬 PC 한 대

Chaos Launcher 1개 + Worker 10개
Launcher가 Worker 10개를 각각 독립 프로세스로 직접 실행
Worker i의 Peer 포트 = 7000 + i
(W1 = 7001, …, W10 = 7010)

연결 (1≤i≤10)	개수	용도
Coordinator ↔ Worker i	10	HELLO, RING, START, HB, REQ, ASSIGN, RESULT, ACK, TAKEOVER, REJOIN, WELCOME, BYE 등 모든 제어 메시지 (원격)
Worker i → Worker (i mod 10 + 1)	10	CKPT 전송 전용 P2P 연결 (로컬). 연결 직후 첫 메시지 PEER i 로 자신을 알림
Launcher ↔ Worker i	10	L→W: CRASH, RESTART / W→L: READY, REJOINED. Worker의 Peer 포트로 접속, 첫 메시지 LAUNCHER

- **하드코딩 금지** IP · 포트 · Worker 번호 · seed는 실행 인자 또는 설정 파일로
- **정식 실행** --step-delay 100 (ms). Launcher의 시간 규칙이 이 값을 기준으로 정해짐
- **제출 로그** 개발은 로컬 한 대에서 해도 되지만, 제출 로그는 반드시 원격 구성에서
- **로그 시각** HH:MM:SS.mmm, 시간대(UTC 또는 KST) 통일하고 Readme에 명시

COORDINATOR

Coordinator의 Thread 구성 — 실행 중 Thread를 새로 만들지 않는다

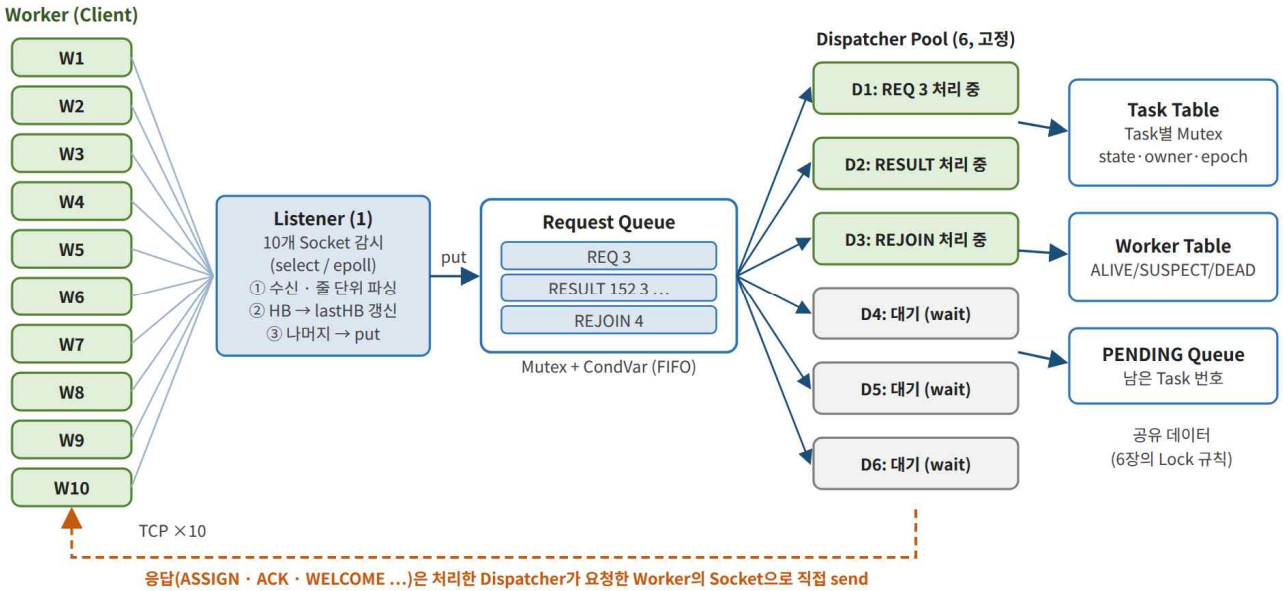
Thread (개수)	하는 일	잠드는 곳	건드리는 공유 데이터
Main (1, 개수 제외)	초기화, Thread 생성, Ring 구성 · START 조율, 종료 대기 → VERIFY · BYE · join · TERMINATE	종료 CondVar	읽기 (검증)
Listener (1)	연결 수락, 10개 Socket을 select/poll/epoll로 감시, 줄 단위 파싱. HB는 직접 처리 (lastHB 갱신), 나머지는 Request Queue로	select 대기	Worker Table, Request Queue
Dispatcher (6, 고정)	Queue에서 HELLO · RING_OK · REQ · RESULT · REJOIN을 꺼내 규칙대로 처리, 응답 전송	Queue CondVar	Task Table, Worker Table, PENDING
Failure Detector (1)	START 전송 후 시작. 1초마다 lastHB 검사 → SUSPECT/DEAD → 다음 ALIVE Worker에게 TAKEOVER	주기 1 sec	Worker Table, Task Table
Monitor (선택, 최대 1)	5초마다 POOL 로그 (Listener의 select 타임아웃으로 대체 가능)	주기 5 sec	읽기

왜 HB는 Listener가 직접 처리하나? Queue에 메시지가 밀리면 HB 반영이 늦어져, 살아 있는 Worker가 DEAD로 오판될 수 있기 때문입니다 (예: Queue 200개 × 30 ms ÷ 6 ≈ 1초 지연).

START를 보낸 시각을 모든 Worker의 lastHB 초기값으로 두고, Failure Detector는 START 이후에 검사를 시작합니다 (그 전에 돌리면 Ring 구성 중 10개가 모두 DEAD).

REQUEST FLOW

Listener · Request Queue · Dispatcher는 어떻게 함께 일하나?



메시지 한 개의 여행 (예: W3의 REQ)

W3 send REQ 3 → Listener 수신·파싱 → Queue에 put(+notify) → 잠든 D1이 깨어나 take → Worker Table Lock 안에서 W3가 DEAD가 아닌지 확인 + PENDING에서 k를 꺼내 Task[k] 배정 → Lock 해제 → ASSIGN k 1 send → D1은 다시 Queue로
※ 같은 Worker의 RESULT와 REQ도 서로 다른 Dispatcher가 동시에 처리할 수 있다 → 그래서 Lock 규칙이 필요

Listener (1) — 입구 직원 1명

도착한 바이트를 줄 단위 메시지로 자름. HB는 lastHB만 갱신, 나머지는 "어느 Socket에서 온 무엇"과 함께 Queue에

Request Queue — 주문표 꽂이

도착 순서대로 쌓는 FIFO. Mutex + Condition Variable로 보호

Dispatcher (6) — 요리사 6명

하나 꺼내 규칙대로 처리 (REQ → 배정, RESULT → Commit, REJOIN → 승인). 응답은 보낸 Worker의 Socket으로 직접

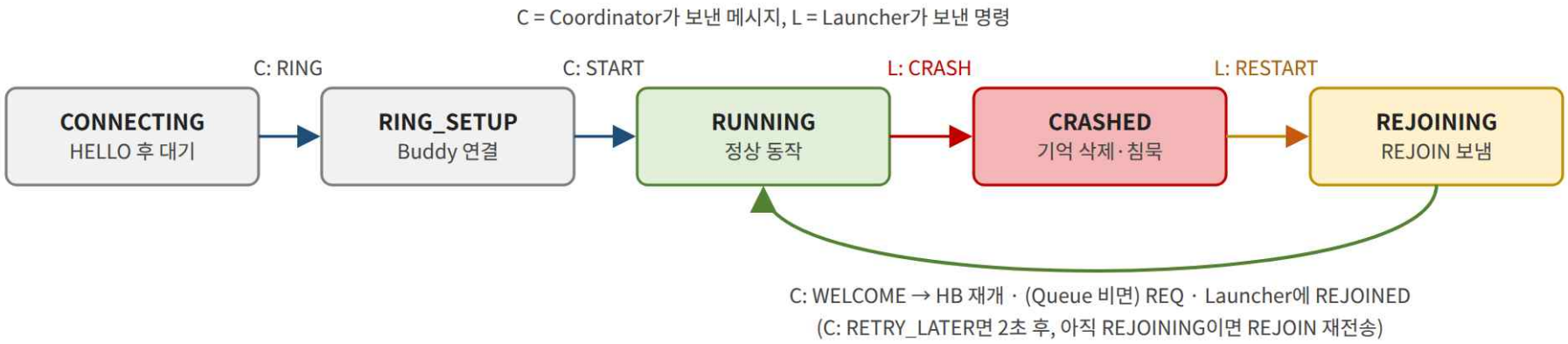
공유 데이터 — 냉장고 · 장부

Task Table, Worker Table, PENDING Queue — Lock으로 보호

WORKER

Worker의 Thread 구성과 내부 상태

Thread (권장, 개수 자유)	하는 일	crash 관련 동작
Control Recv	Coordinator 메시지 수신 (RING, START, ASSIGN, TAKEOVER, ACK, REJECT_STALE, NO_TASK, WELCOME, RETRY_LATER, BYE)	CRASHED면 읽고 버림 (단, BYE는 처리)
Compute	로컬 Queue에서 꺼내 step 계산 → step마다 CKPT → RESULT → Queue가 비면 REQ	Task를 꺼낼 때의 crash_gen을 기억, step 전·전송 직전 비교
Heartbeat	1초마다 HB i	RUNNING이 아니면 CondVar 대기 (침묵)
Peer Server	Peer 포트의 두 종류 연결: 첫 메시지 PEER i → CKPT 수신 / LAUNCHER → CRASH·RESTART 수신	CRASHED면 CKPT 버림, RESTART만 처리



WORKER STATES

Worker 상태별로 하는 일과 보낼 수 있는 메시지

CONNECTING / RING_SETUP	HELLO 후 RING을 기다리고, Buddy에 연결 후 START를 기다림	**HELLO, RING_OK**
RUNNING	정상 동작: HB, 계산, CKPT, RESULT, REQ	**HB, REQ, RESULT, CKPT, (Launcher에) READY · REJOINED**
CRASHED	기억 삭제 후 침묵. 받은 메시지는 버림 (단, Launcher의 RESTART와 Coordinator의 BYE는 처리)	**없음**
REJOINING	REJOIN을 보내고 WELCOME을 기다림. 이 동안 CKPT는 정상 저장, Coordinator 메시지도 RUNNING과 같은 규칙으로 처리	**REJOIN (RETRY_LATER면 2초 후, 아직 REJOINING이면 재전송)**

CRASH 순간 — state = CRASHED, crash_gen + 1 → 로컬 Queue · 진행 중 Task · Checkpoint Store · REQ 대기 플래그 · NO_TASK 플래그를 모두 지우고 침묵 (TCP 연결은 그대로)

CHAOS LAUNCHER

Chaos Launcher — 장애를 무작위로 주입하는 로컬 프로그램

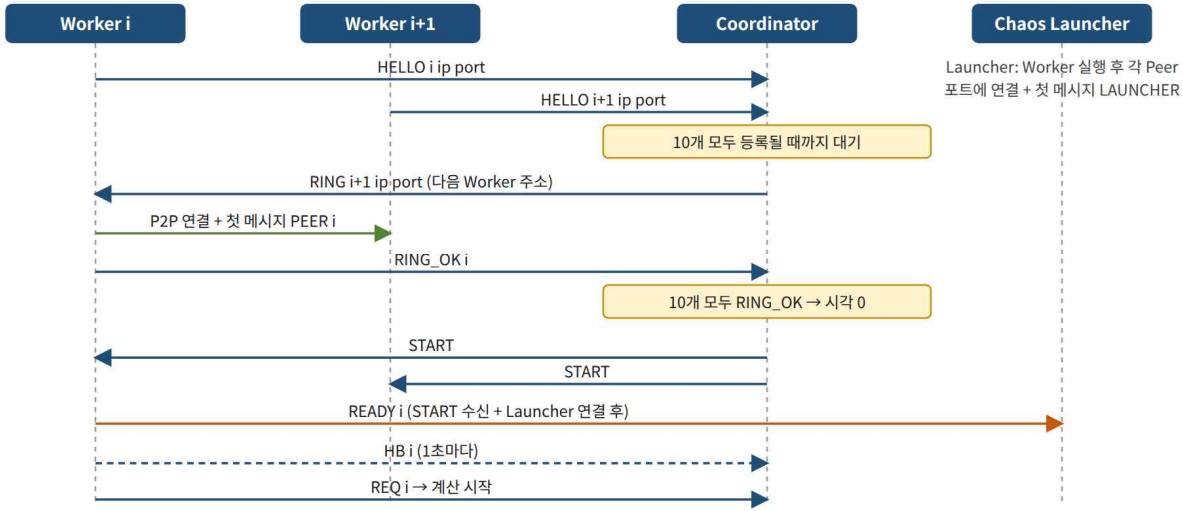
1. Worker 10개 실행
2. 각 Peer 포트에 연결 + 첫 메시지 LAUNCHER (거절되면 0.5초 후 재시도)
3. READY 10개를 받은 시각 = t0 (START 직후, 차이 0.1초 이내), seed 기반 난수로 장애 주입
4. Worker마다 RUNNING / CRASHED / REJOINING / EXITED 기록
5. BYE로 Worker가 종료되면 제어 연결 EOF → EXITED. 10개 모두 EXITED면 Launcher.txt 마감 후 종료

규칙	값 (sec)	이유
victim 선택	그 순간 RUNNING인 Worker 중 무작위	이전에 crash했던 Worker도 다시 뽑힐 수 있다
첫 crash	t0 후 10 ~ 20	계산이 자리 잡은 뒤
crash 간격 I	직전 CRASH 후 5 ~ 15	I < D이면 겹침 → 동시 2개 상황
동시 crash 상한	CRASHED·REJOINING이 2개면 REJOINED까지 대기 → 그 시각부터 I를 새로 뽑음	어느 순간이든 최대 2개
downtime D	CRASH 후 10 ~ 15 경과 시 RESTART	D ≥ 10 → RESTART 전에 DEAD 판정(5~6) 완료
새 crash 중단	t0 후 80 이후 없음	마지막 rejoin(≤ 약 95) 때도 대개 Task가 남음
종료	10개 Worker 모두 EXITED	BYE → 종료 → 제어 연결 EOF

Coordinator는 이 스케줄을 모릅니다. Launcher.txt에 seed와 모든 CRASH · RESTART 시각을 기록해 두고, 나중에 Coordinator 로그의 DEAD · REJOIN과 대조합니다.

STARTUP

기동 순서와 Ring 구성



1 **Launcher가 Worker 10개 실행**

Worker i는 Peer 포트를 먼저 열고 Coordinator에 접속(거절되면 0.5초 후 재시도) → HELLO i ip port. Launcher는 각 Peer 포트에 LAUNCHER

2 **10개 등록 → RING**

Coordinator는 10개가 모두 등록될 때까지 기다린 뒤 RING j ip port (j = i mod 10 + 1)

3 **P2P 연결 → RING_OK**

Worker i는 Worker j의 Peer 포트에 연결해 PEER i를 보낸 뒤 RING_OK i

4 **START (시각 0)**

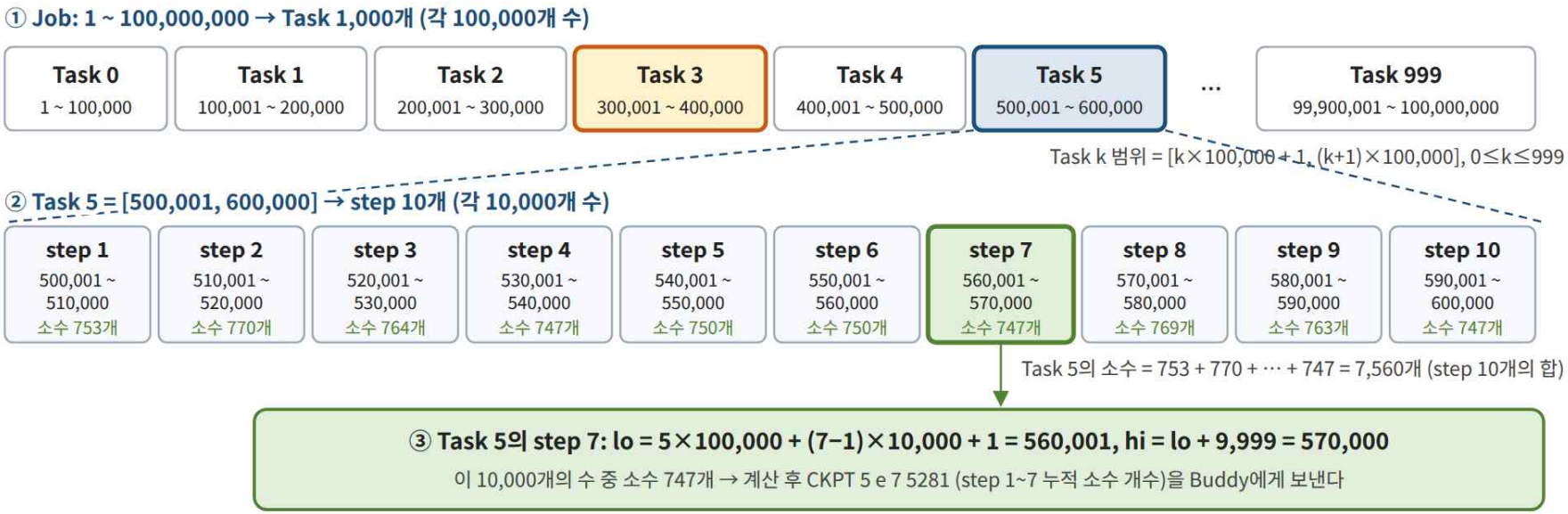
10개 모두 RING_OK → START. Worker는 HB 시작 + REQ. "START 수신 + Launcher 연결" 둘 다 되면 READY i

하나라도 HELLO나 RING_OK를 보내지 않으면 START는 나가지 않습니다.

COMPUTATION

계산 작업 — Job · Task · step

1부터 100,000,000(1억)까지 소수가 몇 개인지 센다 → 큰 범위를 두 번 잘라서 다룬다



Task k (0≤k≤999)

수 100,000개 = 1억 ÷ 1,000

`[k×100,000 + 1, (k+1)×100,000]`

Task 3 = [300,001, 400,000] → 7,863개

step s (1≤s≤10)

수 10,000개 = 100,000 ÷ 10

`lo = k×100,000 + (s−1)×10,000 + 1, hi = lo + 9,999`

Task 5의 step 7 = [560,001, 570,000] → 747개

역으로 n → (k, s)

예: n = 565,432

`k = ⌊(n−1) ÷ 100,000⌋, s = ⌊((n−1) mod 100,000) ÷ 10,000⌋ + 1`

→ k = 5, s = 7

COMPUTATION

왜 Task와 step으로 두 번 자르나?

Task = 나눠 주고 모으는 단위

- Coordinator가 Worker에게 ASSIGN / TAKEOVER로 준다
- 끝나면 RESULT k e v로 반납 → Commit
- Task 결과 1,000개의 합 = 5,761,455
- Task 번호만 주고받는다 (범위는 공식으로 계산)

step = 백업(Checkpoint) 단위

- Worker가 Task 안에서 step 1 → 10 순서로 계산
- step s (1≤s≤9)를 끝낼 때마다 CKPT k e s partial
- partial = step 1~s의 소수 개수 누적합
- crash해도 인계받은 Worker가 step s+1부터 이어서 계산

Task	범위	step별 누적 소수 개수 (step 1 → 10)	결과
0	1 ~ 100,000	1229, 2262, 3245, 4203, 5133, 6057, 6935, 7837, 8713, 9592	9,592
152	15,200,001 ~ 15,300,000	599, 1204, 1796, 2418, 3031, 3642, 4258, 4872, 5481, 6075	6,075
160	16,000,001 ~ 16,100,000	579, 1187, 1794, 2389, 2985, 3560, 4184, 4808, 5399, 5989	5,989
271	27,100,001 ~ 27,200,000	561, 1138, 1720, 2319, 2916, 3507, 4069, 4673, 5238, 5802	5,802

공식 실행: step 하나 ≈ 계산 수 ms + step-delay 100 ms ≈ 0.1초 → Task 하나 ≈ 1초 → 1,000 Task ÷ 10 Worker ≈ 100초 + crash 지연

COUNTING PRIMES

소수 개수 세는 방법

핵심 성질: √n 이하의 소수로만 확인하면 된다

n이 합성수면 n = a × b (2≤a≤b)이고 a ≤ √n. 최대 수가 10^8이므로 $\sqrt{10^8}$ = 10,000 이하의 소수만 있으면 된다 → 10,000 이하 소수 1,229개(마지막 9,973)를 프로그램 시작 시 한 번만 구해 두고(small_primes) 모든 step에서 재사용

방법 A — Trial Division

구간의 수 n마다 small_primes로 작은 것부터 나눈다. p × p > n이면 멈추고 소수로 판정. 이해하기 쉽지만 방법 B보다 느리다 (그래도 100 ms에 비하면 대부분 충분)

방법 B — Segmented Sieve (권장)

[lo, hi]를 모두 "소수 후보"로 표시하고, 각 소수 p의 배수를 지운다. 시작 = max(p×p, lo 이상인 p의 첫 배수) → p 자신을 지우는 실수도 방지. 남은 수가 소수



COUNTING PRIMES

소수 세기 의사코드와 검증 값

```
def make_small_primes(limit = 10000):      # 1,229개, 마지막 9973
    is_p = [True]*(limit+1); is_p[0] = is_p[1] = False
    for p in 2 .. floor(sqrt(limit)):
        if is_p[p]: for m in p*p, p*p+p, … ≤ limit: is_p[m] = False
    return [n for n in 2..limit if is_p[n]]
SMALL = make_small_primes()

def step_range(k, s):                          # 0≤k≤999, 1≤s≤10
    lo = k*100000 + (s-1)*10000 + 1
    return lo, lo + 9999

def count_primes_sieve(lo, hi):                # 방법 B
    mark = [True]*(hi-lo+1)
    for p in SMALL:
        if p*p > hi: break
        start = max(p*p, ceil(lo/p)*p)
        for m in start, start+p, … ≤ hi: mark[m-lo] = False
    if lo <= 1: mark[1-lo] = False             # 1 제외 (Task 0, step 1만)
    return count of True in mark

def count_primes(k, s): return count_primes_sieve(*step_range(k, s))
```

확인 항목	기대 값
small_primes 개수	1,229개, 마지막 9,973
count_primes(0, 1)	1,229 (1을 세면 1,230 → 틀림)
[91, 110] 구간	5 (97, 101, 103, 107, 109)
Task 0 / 152 / 160 / 271	9,592 / 6,075 / 5,989 / 5,802
전체 합	5,761,455 (crash 없이 먼저 확인)
방법 A vs B	같은 구간에서 결과 일치

자주 하는 실수: 1을 소수로 셈 · 구간 끝 hi 누락(양 끝 포함) · 체에서 p 자신을 지움 · 매 step마다 small_primes 재계산

32비트 정수로 충분합니다 (최대 10^8).

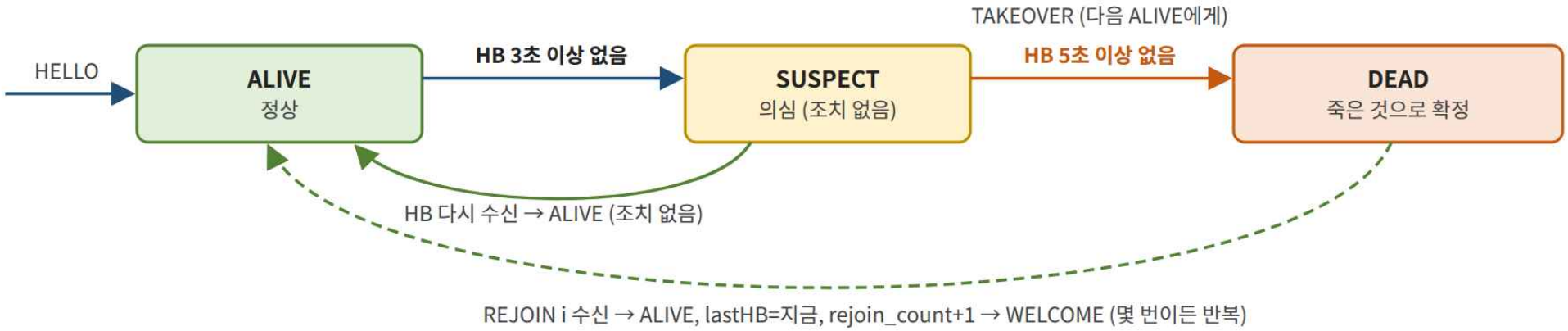
PART 02

장애 내성 메커니즘

Heartbeat → SUSPECT / DEAD → Takeover (+Checkpoint) → Epoch · Commit → Rejoin

FAILURE DETECTION

Heartbeat와 Failure Detector



지금 − lastHB (sec)	판정	하는 일
< 3	ALIVE	없음
3 ≤ … < 5	SUSPECT	로그만. Task는 그대로. HB가 다시 오면 ALIVE 복귀
≥ 5	DEAD	이 Worker가 owner인 모든 ASSIGNED Task를 다음 ALIVE Worker에게 TAKEOVER

꼭 알아 둘 점

- FD가 1초마다 검사 → 감지 지연은 5~6초
- 가상 crash는 연결이 살아 있어 EOF가 없다 → 단서는 오직 HB의 부재
- DEAD 상태 Worker의 HB는 무시 (안전장치)
- 원격에서는 “죽은 것”과 “느린 것”을 구별할 수 없다 → 타임아웃으로 간주

CHECKPOINT

Checkpoint (CKPT) — 진행 상황을 옆 Worker에게 백업

```
CKPT <task k> <epoch e> <완료한 step s> <지금까지 센 소수 개수 partial>     (0≤k≤999, e≥1, 1≤s≤9)
예) CKPT 152 1 6 3642   ← Task 152를 epoch 1로 계산 중, step 6 완료, 누적 3,642개
```

항상 고정 Buddy에게

Worker i는 step s (1≤s≤9)를 끝낼 때마다 Buddy(i mod 10 + 1)에게 보낸다. step 10 뒤에는 CKPT 대신 RESULT

Task별 최신 1개만 보관

비교는 epoch 먼저, 같으면 step (사전식). 새 (e, s)가 더 크면 덮어쓰고, 작거나 같으면 무시

어느 epoch의 CKPT든 사용 가능

소수 개수는 누가 세어도 같으므로, 인계 시 Store의 CKPT를 그대로 이어서 쓴다

Buddy가 crash 중이면 버려짐

Buddy는 읽고 버린다. 보내는 쪽은 신경 쓰지 않는다. 전송 오류도 WARN 후 계속

합계에는 더해지지 않음

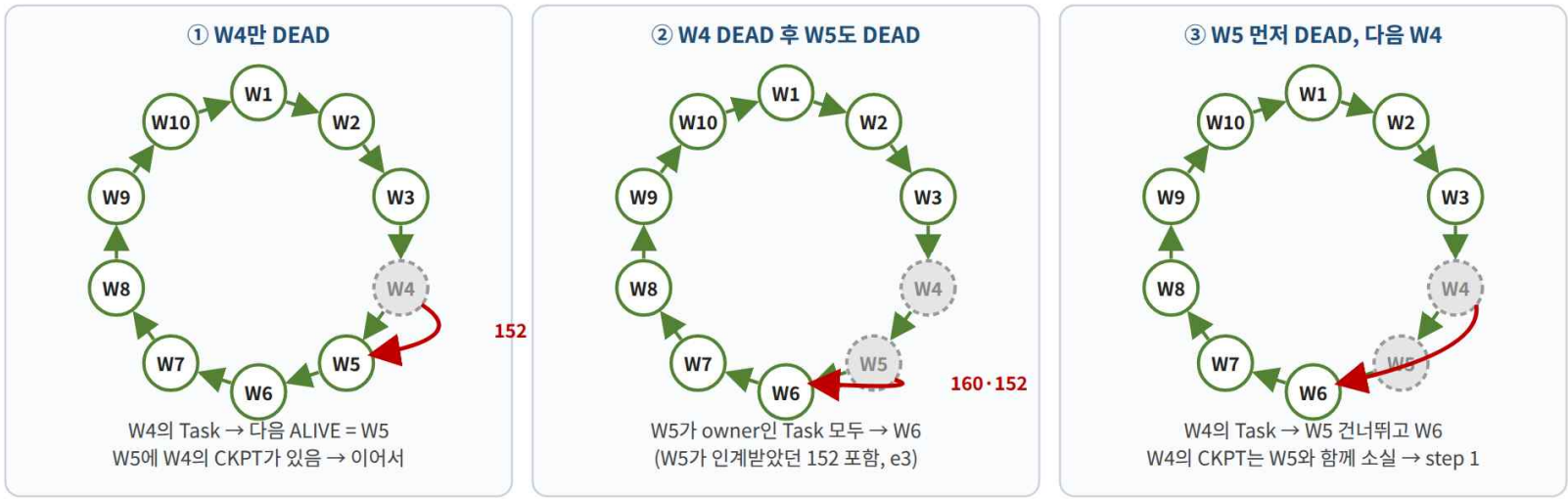
CKPT의 partial은 백업일 뿐. 합계에는 Commit된 RESULT만 들어간다

restart 후에는 빈 Store

crash한 Worker의 Store는 비어 있다. 이후 CKPT부터 다시 쌓는다. 완료된 Task의 CKPT는 지우지 않아도 됨

TAKEOVER

Takeover — 인계 대상은 "링 순서상 다음 ALIVE Worker"



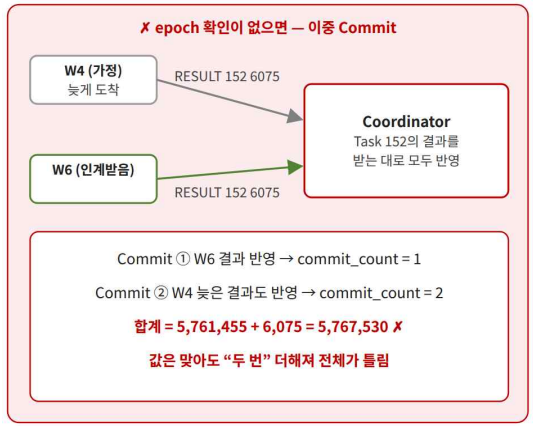
1. 죽은 Worker d의 Task는 d+1, d+2, …(10 다음 1) 순서로 보며 처음 만나는 ALIVE Worker b에게. DEAD와 SUSPECT는 모두 건너뛴다(필수). ALIVE가 없으면 첫 SUSPECT
2. 한 번의 검사에서 여러 Worker가 새로 DEAD면, 먼저 모두 DEAD로 표시한 뒤 번호순으로 인계 대상을 정한다
3. d가 owner인 모든 ASSIGNED Task(이전에 인계받았던 Task 포함)를 epoch+1, owner=b로 바꾸고 TAKEOVER k e from=d
4. b는 Store에 k가 있으면 그다음 step부터, 없으면 step 1부터. d의 Buddy가 함께 crash했다면 CKPT는 없다(③)
5. b가 사실은 막 crash해서 TAKEOVER를 버려도 괜찮다 → b가 DEAD 판정될 때 다시 인계된다(②, epoch 한 번 더 증가)

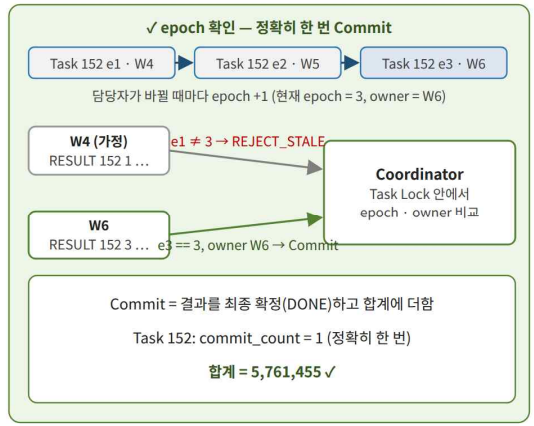
EXACTLY ONCE

Commit · Epoch · 이중 Commit — “정확히 한 번” 반영하기

용어	뜻	이 과제에서
Commit	Task의 결과를 최종 확정 → DONE, 합계에 들어감. 이후 바뀌지 않음	유효한 RESULT → DONE, result 저장, ACK
commit_count	Task가 Commit된 횟수	종료 시 1,000개 모두 정확히 1
owner	지금 그 Task를 맡은 Worker	ASSIGN 때 요청자, TAKEOVER 때 인계 대상
Epoch (e≥1)	담당자가 바뀐 횟수 번호 (Fencing Token)	ASSIGN 때 1, TAKEOVER마다 +1
이중 Commit	같은 Task가 두 번 확정 → 합계가 틀림	반드시 0건 (HW#2 이중예약에 해당)
REJECT_STALE	유효하지 않은 결과를 거부하는 응답	이유: DONE · EPOCH · DEAD

※ 그림의 W4 늦은 RESULT는 규칙을 보여 주기 위한 가정 (W4가 step 10까지 끝내고 RESULT를 보낸 직후 crash한 경우)

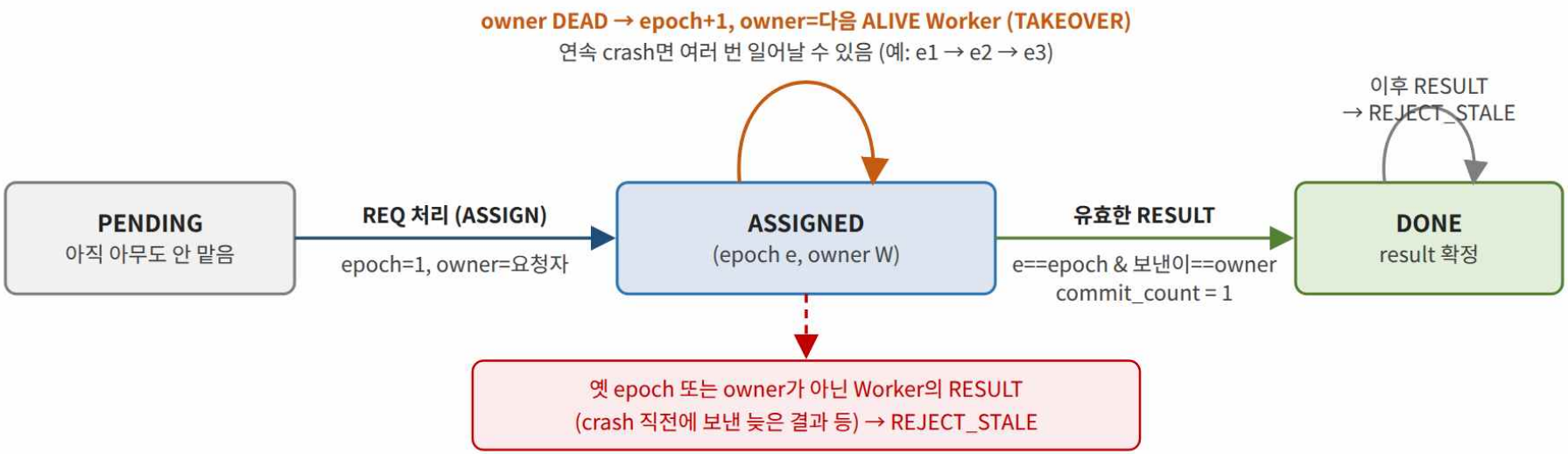




왜 필요한가? crash 직전에 보낸 RESULT가 늦게 도착할 수 있고, 그사이 Task는 이미 다른 Worker에게 넘어갔을 수 있다 → “지금 이 Task의 정당한 담당자가 보낸 결과인가?”를 epoch와 owner로 가린다

호텔 카드키 비유: 주인이 바뀔 때마다 새 카드키 발급, 옛 카드키(옛 epoch)는 문이 안 열린다. 비교와 DONE 표시는 반드시 Task Lock 안에서 한 번에!

Task 상태 전이와 Rejoin



Task 상태 전이 요점

- PENDING → (REQ 처리, ASSIGN) → ASSIGNED (epoch 1)
- owner DEAD → epoch+1, owner = 다음 ALIVE (여러 번 가능: e1→e2→e3)
- DONE으로 가는 길은 "유효한 RESULT" 하나뿐
- DONE 이후의 RESULT, 옛 epoch의 RESULT → REJECT_STALE

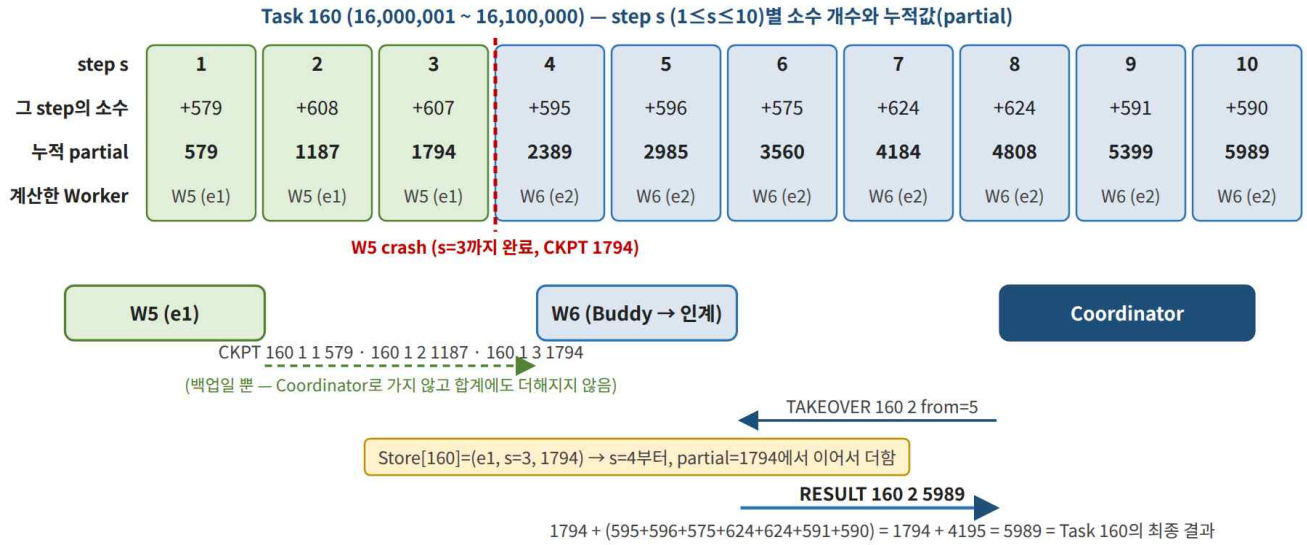
Rejoin 절차

- Launcher의 RESTART → Worker는 REJOINING → REJOIN i
- Coordinator: i가 DEAD면 ALIVE, lastHB=지금, rejoin_count+1 → WELCOME / 아직 ALIVE·SUSPECT면 RETRY_LATER (2초 후 재시도)
- Worker: WELCOME → RUNNING, HB 재개, Launcher에 REJOINED, Queue가 비어 있으면 REQ
- 재합류한 Worker는 이전 Task를 전혀 모른다 (늦은 ACK·REJECT는 무시)

TOTAL SUM

합계 계산 (1) — 한 Task의 결과가 만들어지는 과정

메시지	어디로	담긴 값	합계에 들어가나?
CKPT k e s partial	Worker i → Buddy(i) (Coordinator를 거치지 않음)	step 1~s의 중간 누적값 (1≤s≤9)	아니오 — 인계받은 Worker의 출발점
RESULT k e v	Worker → Coordinator	step 1~10 전체 최종값 v	유효하면 예 (Commit) / 아니면 거부



읽는 법

- partial = 이 Task에서 지금까지 센 소수 개수
- 인계받은 Worker는 CKPT의 (s, partial)에서 시작해 s+1 ~ 10을 더한다
- CKPT가 없으면 s=1, partial=0부터 (예: Task 152)
- W5가 세던 1794는 어디에도 더해지지 않고 W6 계산의 "출발점"으로만 쓰인다
- 누가 세어도 소수 개수는 같으므로 최종값은 항상 5989

TOTAL SUM

합계 계산 (2) — Coordinator의 결과 장부

Dispatcher가 처리한 RESULT (도착 순서)

① **RESULT 0 1 9592 (W1)**
e1=1, owner W1 → Commit → result[0]=9592

② **RESULT 160 2 5989 (W6)**
e2=2, owner W6 → Commit → result[160]=5989

③ **RESULT 152 3 6075 (W6)**
e3=3, owner W6 → Commit → result[152]=6075

④ **(가정) RESULT 152 1 6075 (W4의 늦은 결과)**
W4가 DEAD면 DEAD, 아니면 이미 DONE → REJECT_STALE — 합계 +0

⑤ **RESULT 160 2 5989 (같은 결과가 또 도착했다고 가정)**
이미 DONE → REJECT_STALE (DONE) — 합계 변화 없음

(참고) CKPT 160 1 3 1794
Coordinator로 오지 않음 → Worker 간 백업 — 합계와 무관

Coordinator의 Task Table (결과 장부, 발췌)

k	state	epoch	owner	result	commit
0	DONE	1	W1	9592	1
…	…	…	…	…	…
152	DONE	3	W6	6075	1
160	DONE	2	W6	5989	1
271	ASSIGNED	1	W4	—	0
…	…	…	…	…	…
999	PENDING	0	—	—	0

Commit될 때만 result가 채워지고 commit_count가 1이 된다

종료 시 Main Thread의 VERIFY: 합계 = Σ result[k] (0≤k≤999, 모두 DONE) = 5,761,455
그리고 Σ commit_count[k] = 1,000 (모든 Task가 정확히 1) — 거부된 RESULT와 CKPT는 합계에 들어가지 않는다

#	도착한 RESULT	판정 (Task Lock 안)	합계
①	RESULT 0 1 9592 (W1, 정상)	e=1=epoch, owner=W1	+9,592
②	RESULT 160 2 5989 (W6, CKPT로 이어서)	e=2=epoch, owner=W6	+5,989
③	RESULT 152 3 6075 (W6, 두 번 인계)	e=3=epoch, owner=W6	+6,075
④	(가정) RESULT 152 1 6075 (W4의 늦은 결과)	W4가 아직 DEAD면 DEAD, 아니면 이미 DONE 또는 e=1≠3	+0 (거부)
⑤	(가정) RESULT 160 2 5989 (중복 도착)	이미 DONE	+0 (거부)
⑥	RESULT 271 1 5802 (W4, rejoin 후)	e=1=epoch, owner=W4	+5,802

합계 방법	구현	주의
A. 종료 시 한 번에 (권장)	Main이 1,000개 DONE 확인 후 Σ result[k], Σ commit_count = 1,000 확인 → VERIFY	모든 Dispatcher가 멈춘 뒤 읽으므로 경합 없음
B. Commit할 때마다 누적	Commit 직후 total += v, done_count += 1 (POOL 로그용)	total도 Mutex로 보호. 종료 시 A와 같아야 함

정리: "유효한 RESULT만, Task Lock 안에서, 한 번만 Commit" → 빠짐없음(1,000 DONE) + 중복 없음(commit_count 1) → 5,761,455

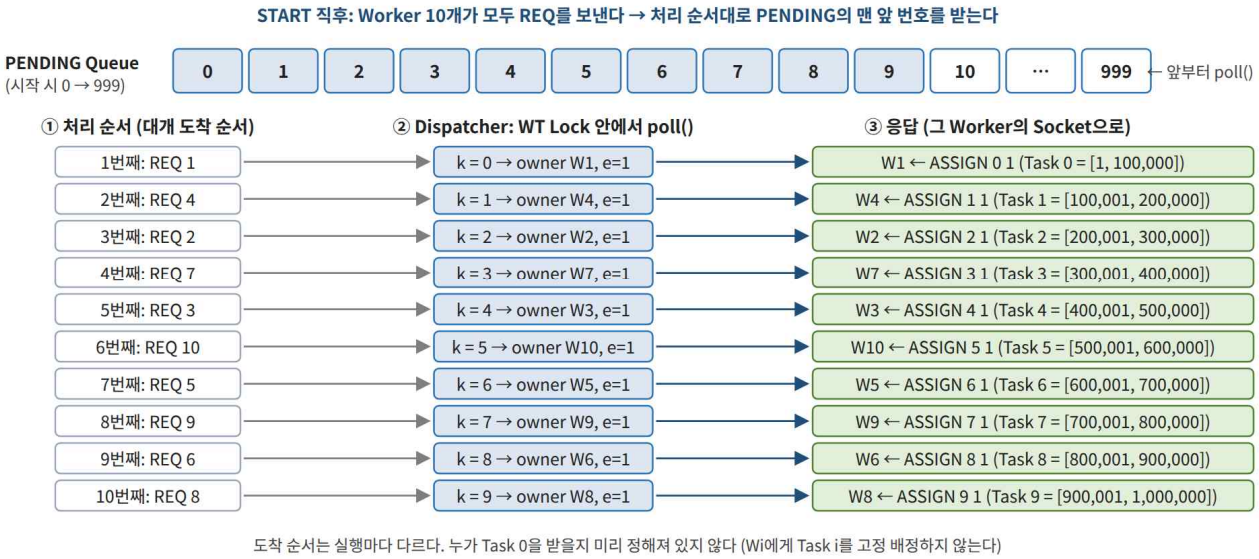
PART 03

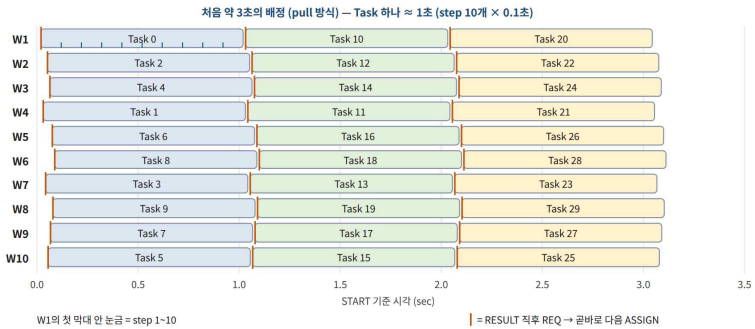
Task 흐름 따라가기

최초 배정 → 계산 → 반납, 그리고 crash 시 Buddy가 인계받아 둘 다 끝내기

TASK FLOW ①

최초 배정 — Worker가 요청(REQ)해야 Task를 받는다





시작 상태

Task Table 1,000칸을 모두 PENDING(epoch 0)으로, PENDING Queue에 0, 1, ..., 999를 오름차순으로

첫 REQ

START를 받으면 로컬 Queue가 비어 있으므로 REQ i를 한 번. 10개가 Request Queue에 쌓인다

배정

Worker Table Lock 안에서 ① DEAD 아닌지 확인 ② PENDING 맨 앞 k ③ Task[k] = ASSIGNED, owner=i, e=1 → Lock 해제 후 ASSIGN k 1

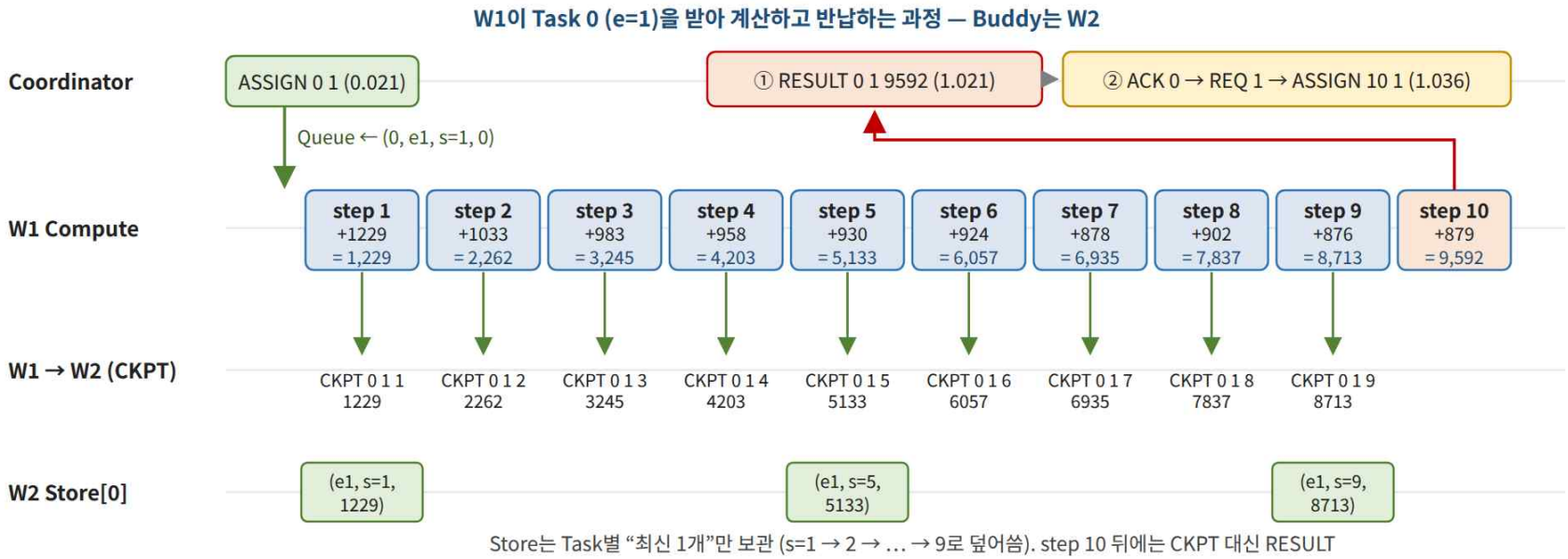
고정 배정 아님

먼저 poll()한 REQ가 작은 번호 (대개 도착 순서, 실행마다 다름). poll()이 Lock 안이라 같은 번호가 두 번 나가지 않음

pull 방식: 받은 Task를 끝내 Queue가 비면 다시 REQ → 그 순간 PENDING의 맨 앞 번호. 빨리 끝낸 Worker가 더 많이 가져가 부하가 저절로 나뉜다

TASK FLOW ②

Worker 안에서 — Task 0 계산과 RESULT 반납 (W1, Buddy W2)



1 **ASSIGN 0 1**

Control Recv가 Queue에 (0, e1, s=1, 0)

2 **step 1 ~ 10 계산**

step_range(0, s)로 구간을 구해 센다

3 **CKPT → W2**

step 1~9마다 CKPT 0 1 s partial

4 **W2 Store[0]**

최신 1개로 덮어쓰기만 (계산에 안 씀)

5 **RESULT 0 1 9592**

step 10 뒤에는 CKPT 대신 RESULT

6 **Commit → ACK 0**

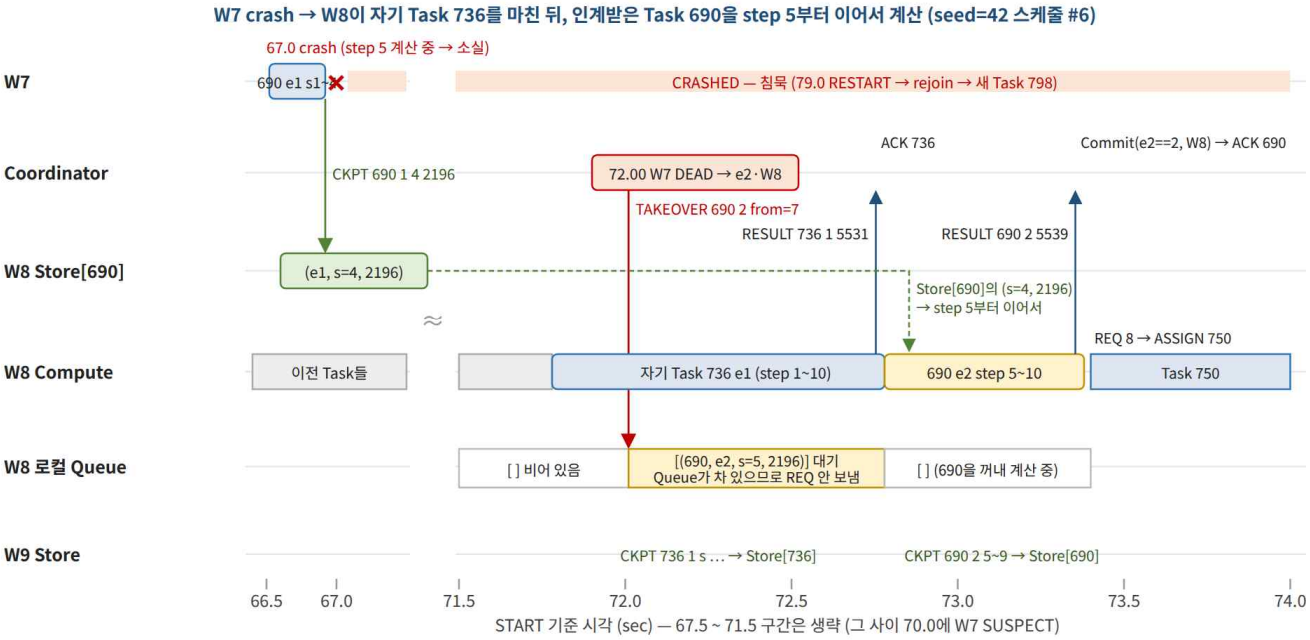
Task[0] Lock 안에서 확인 후 확정

7 **REQ 1 → ASSIGN 10**

Queue가 비었으니 다음 Task 요청

TASK FLOW ③

crash 시 — Buddy가 두 Task를 모두 끝내기 (W7 crash)



시각	일어나는 일
66.92	W7: Task 690 step 4 완료 → CKPT 690 1 4 2196 → W8 Store
67.00	W7 CRASH (step 5 계산 소실), 침묵 · 연결 유지
72.00	W7 DEAD → 690: e2, owner W8 → TAKEOVER 690 2 from=7
72.01	W8 Queue ← (690, e2, 시작 step 5, 2196). 736 계속 계산, REQ 안 보냄
72.78	W8: RESULT 736 1 5531 → Commit
73.38	W8: 690 step 5~10 → RESULT 690 2 5539 → Commit
73.39	Queue가 비었으므로 REQ 8 → ASSIGN 750
79.0~	W7 rejoin → 새 Task 798 (690은 모름)

한 줄로 들어갈 뿐

TAKEOVER는 Queue에 (k, e, 시작 step, partial) 추가. 진행 중 Task를 끝낸 뒤 처리 (권장)

시작점 = Store

Store[690] = (s=4, 2196) → step 5부터

새 epoch 사용

CKPT 690 2 s …는 W9에게, RESULT도 e2

REQ는 Queue가 빌 때만

736, 690 모두 끝낸 뒤 REQ

PART 04

동기화 설계와 통신 프로토콜

Mutex · Lock Ordering · Condition Variable · 메시지 명세와 처리 규칙

SYNCHRONIZATION

공유 자원과 보호 방법 — 동기화가 틀리면 틀린 결과가 나올 수 있다

공유 자원	쓰는 Thread	읽는 Thread	보호 방법
Task Table [0..999]	Dispatcher(ASSIGN, RESULT), FD(TAKEOVER)	Main(VERIFY), Monitor	Task마다 Mutex (전체 Lock 1개 금지)
Worker Table [1..10]	Listener(lastHB), Dispatcher(HELLO, REJOIN), FD(상태)	Dispatcher(REQ 확인), Monitor	Worker Table Lock 1개
Request Queue	Listener(put)	Dispatcher(take)	Mutex + CondVar(not_empty)
PENDING Queue	Dispatcher(poll)	—	thread-safe Queue 또는 Mutex
done_count / 종료	Dispatcher(+1)	Main(대기)	Mutex + CondVar(all_done)
Coordinator의 Worker Socket 송신	여러 Dispatcher · FD가 같은 Worker에게 동시 send	—	Worker Socket마다 송신 Lock
Worker: state	Peer Server(CRASH·RESTART), Control Recv(WELCOME, BYE)	HB, Compute, Control Recv	Mutex + CondVar
Worker: 로컬 Queue · Store	Control Recv(ASSIGN, TAKEOVER), Peer Server(CKPT), CRASH(비움)	Compute, Control Recv	Mutex (+CondVar)

Coordinator Lock 순서

Worker Table Lock → Task Lock (번호 오름차순, 가능하면 하나씩)

Worker Lock 순서

q_lock → state_lock → store_lock (하나씩 잡고 풀면 순서 문제 없음)

Lock 잡은 채 send 금지

Lock 안에서는 상태만 바꾸고, 보낼 내용은 변수에 담아 Lock을 푼 뒤 send · 로그

EXAMPLE ① MUTEX

이중 Commit 막기 — 확인과 변경을 한 Lock 안에서 (check-then-act)

가정 (가상 예 — S5와는 다른 상황): W5가 crash 직전에 RESULT 160 1 5989를 보냈다. 이것을 꺼낸 D2가 확인까지만 하고 CPU를 빼앗긴 사이, W5가 DEAD → Task 160이 W6에게 인계(e2) → W6의 RESULT 160 2 5989를 D5가 처리

✗ 잘못된 구현

```
t = task[160]
if t.state != DONE and t.epoch == e \
   and t.owner == sender:      # 확인
    # ← 여기서 다른 Thread가 끼어듦
    t.state = DONE             # 변경
    t.commit_count += 1
```

✓ 올바른 구현

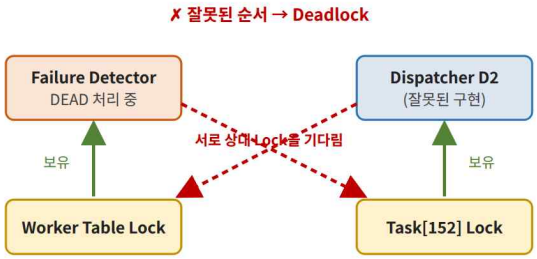
```
# (보낸 Worker가 DEAD면 먼저 REJECT(DEAD) — 38쪽 규칙)
with task[160].lock:
    t = task[160]
    if t.state == DONE:        reply = REJECT(DONE)
    elif t.epoch != e or t.owner != sender:
                               reply = REJECT(EPOCH)
    else: t.state = DONE; t.result = v
          t.commit_count += 1; reply = ACK
send(sender, reply)            # Lock 밖에서 전송
```

순서	Thread	잘못된 구현에서 일어나는 일
1	D2 (W5의 e1)	확인: DONE 아님, epoch 1 == 1, owner W5 → 통과. 변경 직전에 CPU를 빼앗김
2	FD	W5 DEAD → Task 160: epoch 2, owner W6 → TAKEOVER
3	D5 (W6의 e2)	확인 통과 → DONE, commit_count = 1
4	D2	1번의 확인을 믿고 DONE, commit_count = 2 → 합계에 5,989가 두 번 ✗

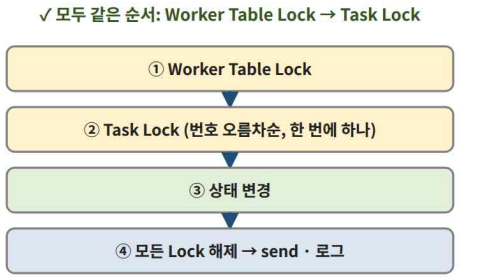
올바른 구현: D2가 먼저 Lock을 잡으면 W5 결과로 Commit, 이후 FD는 DONE Task를 건너뜀. FD가 먼저면 D2는 epoch 불일치로 REJECT. 어느 순서든 정확히 한 번!

EXAMPLE ② ③

Deadlock 막기와 "고아 Task" 막기



FD: WT → Task 순서 / D2: Task → WT 순서 = 순환 대기



먼저 WT를 잡은 쪽이 끝낼 때까지 다른 쪽은 WT 앞에서 기다린다 → 순환이 생길 수 없음

Deadlock 시나리오

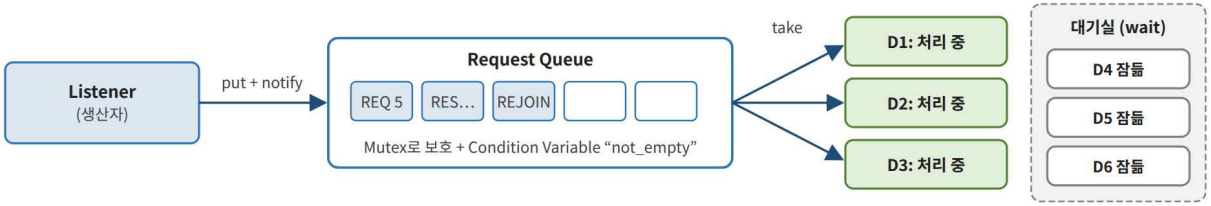
- t1: FD가 Worker Table Lock, D2가 Task[152] Lock 획득
- t2: FD는 Task[152]를, D2는 "W4가 DEAD인가?" 보려고 WT Lock을 요청
- t3: 서로 영원히 대기 → FD 멈춤 → 이후 어떤 crash도 감지 못함
- 해결: 모두 WT → Task 순서. RESULT에서 Worker 상태는 Task Lock 전에 본다

순서	✗ 잘못된 구현 (확인과 배정이 분리)	✓ 올바른 구현 (WT Lock 안에서 확인 + 배정)
1	D3: W5가 DEAD 아님 확인 → 통과 (Lock 해제)	D3: WT Lock 획득 → W5 DEAD 아님 확인
2	FD: W5 DEAD, W5 소유 Task 검색 (161은 아직 PENDING → 없음)	FD: WT Lock 요청 → 대기
3	D3: Task 161을 owner=W5로 ASSIGN	D3: Task[161] Lock → ASSIGN(owner W5) → 모든 Lock 해제
4	161은 DEAD인 W5에게 남음 → 999/1000에서 영원히 멈춤	FD: W5 DEAD → 소유 Task 검색 → 161도 발견 → TAKEOVER ✓

③ 원자적 배정: W5가 crash 직전에 보낸 REQ 5를 D3이 처리하는 순간 W5의 DEAD 처리가 겹친 경우

EXAMPLE ④ CONDVAR

Condition Variable — 바쁜 대기(busy-waiting) 대신 잠들기



Queue가 비면 Dispatcher는 CPU를 쓰지 않고 “대기실”에서 잠든다(wait). Listener가 put 후 notify하면 하나가 깨어나 Lock을 다시 잡고 take한다.

이 과제의 CondVar	기다리는 조건 (while)	누가 깨우나
Request Queue	Queue가 비어 있음	Listener의 put / 종료 시 STOP
종료 대기 (Main)	done_count < 1000	1000번째 Commit을 한 Dispatcher
Worker 로컬 Queue	Queue가 비어 있음 and not stopping	ASSIGN · TAKEOVER, WELCOME · BYE
Worker state	state != RUNNING and not stopping	WELCOME 또는 BYE (notify_all)

```
# Request Queue (Python threading 예)
lock = threading.Lock()
not_empty = threading.Condition(lock)
q = deque()

def put(item):                    # Listener
    with not_empty:
        q.append(item)
        not_empty.notify()        # 한 명 깨우기

def take():                       # Dispatcher
    with not_empty:
        while not q:              # if가 아니라 while!
            not_empty.wait()      # Lock 풀고 잠듦
        return q.popleft()
```

왜 while? 깨어났을 때 다른 Thread가 이미 가져갔을 수 있고, 이유 없이 깨는 spurious wakeup도 있다. sleep 반복 확인은 감점!

PROTOCOL

통신 메시지 — 줄바꿈(\n) 단위, 형식은 자유 (그대로 써도 됨)

방향	메시지	의미 / 보내는 시점
W → C	HELLO i ip port	처음 접속. ip · port = 이 Worker의 Peer 주소 (예: 127.0.0.1, 7000+i)
	RING_OK i / HB i	Buddy에 P2P 연결 완료 / RUNNING일 때 1초마다
	REQ i	로컬 Queue가 비었을 때 (응답 전 재전송 금지)
	RESULT k e v	Task k를 epoch e로 끝냄, 결과 v (예: RESULT 152 3 6075)
	REJOIN i	RESTART 후 재합류 요청
C → W	RING j ip port / START	너의 Buddy 주소 / 10개 모두 RING_OK → 계산 시작
	ASSIGN k e / NO_TASK	REQ 응답 (e는 항상 1) / 줄 Task 없음
	TAKEOVER k e from=d	죽은 d의 Task k를 새 epoch e로 맡아라 (e≥2)
	ACK k / REJECT_STALE k reason	반영 완료 / 거부 (DONE, EPOCH, DEAD)
	WELCOME / RETRY_LATER / BYE	재합류 승인 / 아직 DEAD 전 → 2초 후 다시 / 전체 종료
W → W	PEER i / CKPT k e s partial	Buddy 포트 연결 직후 첫 메시지 / step 1~9 완료 시
L ↔ W	LAUNCHER, CRASH, RESTART / READY i, REJOINED i	Launcher 제어 (Worker Peer 포트 경유)

메시지 경계 처리

TCP는 바이트 스트림 → recv 한 번에 여러 메시지가 붙거나 잘려 온다. Socket마다 버퍼를 두고 \n 단위로 자른다

```
recv#1 "HB 3\nREQ 3\nRES"
recv#2 "ULT 152 1 6075\n"
```

보낸 Worker는 Socket으로 판단

Coordinator는 메시지 안의 번호가 아니라 도착한 Socket으로 보낸 Worker를 판단한다 (HELLO 때 Socket ↔ 번호 등록). 그래서 번호 없는 RESULT도 owner와 비교할 수 있다. HB i, REQ i의 번호는 로그 · 검증용

PROTOCOL

Coordinator의 메시지 처리 규칙 (조건은 위에서부터 순서대로)

메시지	조건	응답	처리
HELLO	처음 보는 번호(1~10) / 중복 · 범위 밖 · START 이후	(대기) / 없음	등록. 10개가 모이면 RING / 무시 (WARN)
RING_OK	—	(대기)	10개 모두 오면 START
HB (Listener)	ALIVE · SUSPECT / DEAD	없음	lastHB 갱신 (SUSPECT면 ALIVE로) / 무시
REQ i	i가 DEAD (WT Lock 안에서 확인)	없음	무시 (WARN)
	PENDING 있음	ASSIGN k 1	같은 Lock 구간에서 Task Lock → 배정 (SUSPECT도 배정)
	PENDING 없음	NO_TASK	Worker는 이후 REQ 없이 TAKEOVER · BYE를 기다림
RESULT k e v	보낸 Worker가 DEAD	REJECT_STALE (DEAD)	
	Task k 이미 DONE	REJECT_STALE (DONE)	중복 결과
	e ≠ epoch 또는 보낸이 ≠ owner	REJECT_STALE (EPOCH)	인계 전 옛 결과
	그 외	ACK k	DONE, result, commit_count + 1
REJOIN i	i가 DEAD / ALIVE · SUSPECT	WELCOME / RETRY_LATER	ALIVE, lastHB = 지금, rejoin_count + 1
DEAD 처리 (FD)	owner = d인 ASSIGNED Task	TAKEOVER	epoch + 1, owner = 다음 ALIVE. 새 DEAD가 둘이면 먼저 둘 다 DEAD 표시 후 번호순

PROTOCOL

Worker의 메시지 처리 규칙

CRASHED 상태에서는 RESTART와 BYE 외에 모두 버림 · REJOINING 상태에서는 RUNNING과 같게 처리

받은 메시지	처리
PEER i / LAUNCHER	그 연결을 "선행 Worker i의 CKPT 연결" 또는 "Launcher 제어 연결"로 기록 (Peer 포트의 첫 메시지)
RING j ip port	Buddy j의 Peer 포트에 연결 → RING_OK i
START	RUNNING → HB 시작, REQ. Launcher 연결이 있으면 즉시, 없으면 연결되는 순간 READY
ASSIGN k e	로컬 Queue에 (k, e, step 1, 0) 추가, REQ 대기 해제
TAKEOVER k e from=d	Store에 k가 있으면 (k, e, 저장 step+1, 저장 partial), 없으면 (k, e, 1, 0)을 Queue에 추가. CKPT의 epoch는 달라도 됨
ACK / REJECT_STALE	로그 · 통계. 모르는 Task면 무시
NO_TASK	NO_TASK 플래그 → 이후 REQ 금지, TAKEOVER · BYE만 기다림. crash하면 지워지므로 rejoin 후 REQ 한 번
CRASH (Launcher)	state = CRASHED, crash_gen + 1 → Queue · 진행 중 Task · Store · REQ 대기 · NO_TASK 플래그 모두 지움 → 침묵
RESTART (Launcher)	state = REJOINING → REJOIN i
WELCOME	state = RUNNING (notify_all) → HB 재개, REJOINED, 로컬 Queue가 비어 있으면 REQ
RETRY_LATER	2초 후 REJOIN 재전송 (타이머 등으로, 그때도 state가 REJOINING일 때만)
CKPT k e s p (Peer)	Store에 없거나 (e, s)가 사전식으로 더 크면 저장
BYE	요약 로그 후 종료 (CRASHED에서도). stopping = True, notify_all로 모든 대기 Thread를 깨워 끝냄

IMPLEMENTATION

Worker 의사코드 (1) — crash_gen으로 “crash 이전 Task”를 구별

```
def on_crash():                  # Peer Server가 CRASH 수신
    with state_lock: state = CRASHED; crash_gen += 1
    with q_lock: local_q.clear(); requested = False; no_task = False
    with store_lock: store.clear()

def send_if_alive(gen, target, msg):   # 확인과 전송 사이에 crash가 끼지 않게
    with state_lock:
        if state != RUNNING or crash_gen != gen: return False
    try: target.send(msg)
    except SendError: log WARN     # Buddy(CKPT) 오류만 계속, Coordinator 오류는 치명적
    return True

def take_task():                 # 꺼내는 순간의 crash_gen을 함께 반환
    while True:
        wait_until_running_or_stop()
        with q_lock:
            with state_lock: running, gen = (state == RUNNING), crash_gen
            if stopping: exit_thread()
            if running and local_q: return local_q.popleft() + (gen,)
            need_req = running and not local_q and not requested and not no_task
            if need_req: requested = True
            else: q_cv.wait()
        if need_req: send_if_alive(gen, coord, f"REQ {i}")

def compute_loop():
    while True:
        (k, e, s0, partial, gen) = take_task()
        for s in s0..10:
            if crash_gen != gen: break
            partial += count_primes(k, s); sleep(step_delay)
            if s < 10 and not send_if_alive(gen, buddy, f"CKPT {k} {e} {s} {partial}"): break
        else: send_if_alive(gen, coord, f"RESULT {k} {e} {partial}")
```

왜 crash_gen?

step 하나는 계산 + 100 ms. 시작 전에 RUNNING을 확인해도 그 사이 crash가 올 수 있다 → 보내기 직전에 다시 확인

왜 “꺼내는 순간”에 읽나?

기다리기 전에 읽으면, 기다리는 동안 crash → rejoin → ASSIGN이 지나간 새 Task를 옛 gen으로 비교해 버린다 → 999/1000에서 멈춤

Lock 순서

q_lock → state_lock → store_lock. on_crash처럼 하나씩 잡고 풀면 문제 없음

종료

BYE → stopping = True + notify_all. HB·Compute의 대기 조건에 not stopping → CRASHED에서도 끝남

IMPLEMENTATION

Worker 의사코드 (2) — 메시지 수신 핸들러 (requested 해제 · notify)

```
def on_assign(k, e):                  # Control Recv — REQ의 응답
    with q_lock:
        local_q.append((k, e, 1, 0))
        requested = False              # ★ 내리지 않으면 다음 REQ를 영영 못 보냄
        q_cv.notify_all()

def on_takeover(k, e, d):             # Control Recv — 인계받은 Task
    with q_lock:                      # q_lock → store_lock 순서
        with store_lock: ck = store.get(k)    # (epoch, step, partial) 또는 없음
        local_q.append((k, e, ck.step + 1, ck.partial) if ck else (k, e, 1, 0))
        q_cv.notify_all()

def on_no_task():                     # Control Recv
    with q_lock: no_task = True; requested = False

def on_restart():                     # Peer Server가 Launcher의 RESTART 수신
    with state_lock: state = REJOINING
    send_coord(f"REJOIN {i}")

def on_welcome():                     # Control Recv
    with state_lock: state = RUNNING; cv.notify_all()   # HB · Compute 깨우기
    send_launcher(f"REJOINED {i}")
    with q_lock: q_cv.notify_all()    # Queue가 비었으면 take_task()가 REQ

def on_bye():                         # Control Recv (CRASHED에서도 처리)
    with state_lock: stopping = True; cv.notify_all()
    with q_lock: q_cv.notify_all()
    log TERMINATE; close_all_sockets()
```

requested 플래그

REQ를 보낼 때 True, ASSIGN · NO_TASK · CRASH 때 False. ASSIGN에서 내리는 것을 빠뜨리면 첫 Task 뒤에 멈춘다

REQ 시점

(1)의 take_task는 Queue가 비는 즉시 보내는 "RESULT 직후" 방식. ACK 뒤에 보내려면 ACK 수신 때 requested를 확인해 notify하도록 바꾼다 (둘 다 허용, Readme에)

RETRY_LATER

2초 후 REJOIN 재전송. Control Recv를 막지 말고 타이머 등에서, 그때도 state가 REJOINING일 때만 (WELCOME 뒤 중복 REJOIN 금지)

HB Thread

state != RUNNING and not stopping 동안 cv.wait(). WELCOME · BYE의 notify_all로 깨어남

IMPLEMENTATION

Coordinator 의사코드 핵심 — Failure Detector · REQ · REJOIN

```
# ----- Failure Detector -----
loop every 1s:
    moved = []; newly_dead = []
    with wt_lock:                                  # ① Worker Table Lock
        for w in workers if w.state in (ALIVE, SUSPECT):
            gap = now() - w.lastHB
            if gap >= 5: newly_dead.append(w)
            elif gap >= 3: w.state = SUSPECT
        for d in newly_dead: d.state = DEAD         # 먼저 모두 DEAD 표시
        for d in sorted(newly_dead):                # 그다음 번호순 인계
            b = next_alive(d)                       # DEAD·SUSPECT 건너뜀
            for k in 0..999:                        # ② Task Lock, 오름차순
                with task[k].lock:
                    if task[k].state == ASSIGNED and task[k].owner == d:
                        task[k].epoch += 1; task[k].owner = b
                        moved.append((b, k, task[k].epoch, d))
    for (b, k, e, d) in moved: send(b, f"TAKEOVER {k} {e} from={d}")   # ③ Lock 밖

# ----- Dispatcher: REQ / REJOIN -----
def on_req(i):
    with wt_lock:                                  # 확인 + 배정을 한 구간에서
        if workers[i].state == DEAD: log WARN; return
        k = pending.poll()
        if k is not None:
            with task[k].lock: task[k].state, task[k].owner, task[k].epoch = ASSIGNED, i, 1
    send(i, f"ASSIGN {k} 1" if k is not None else "NO_TASK")

def on_rejoin(i):
    with wt_lock:
        ok = workers[i].state == DEAD
        if ok: workers[i].state = ALIVE; workers[i].lastHB = now(); workers[i].rejoin_count += 1
    send(i, "WELCOME" if ok else "RETRY_LATER")
```

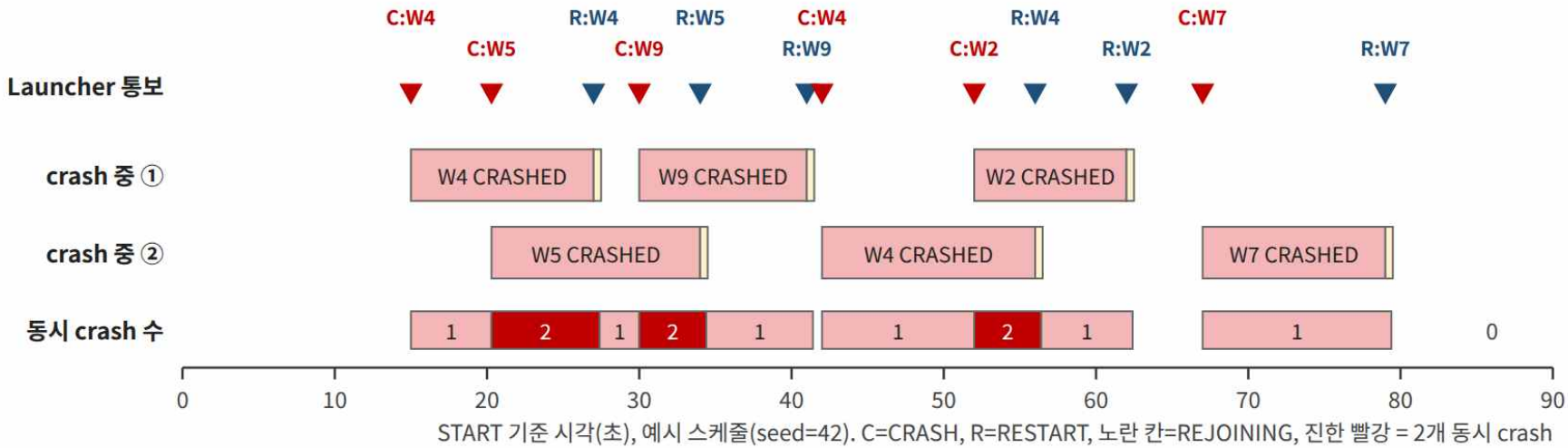
PART 05

시나리오별 동작 예제

seed=42 한 번의 실행 — S1 ~ S10

SCENARIOS

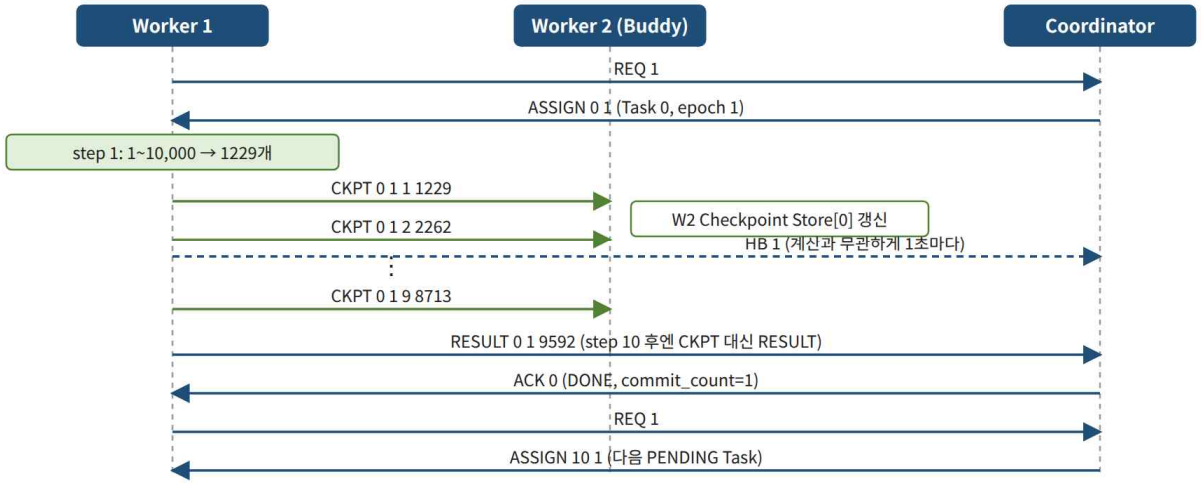
crash 스케줄 예시 (seed = 42) — 시각은 START(≈ t0) 기준



#	victim	CRASH (sec)	RESTART (sec)	D / I (sec)	그 순간 crash 중인 다른 Worker · 특징
1	W4	15.0	27.0	12.0 / —	없음 (첫 crash)
2	W5	20.3	34.0	13.7 / 5.3	W4 (이웃) → S5 이웃 동시 crash
3	W9	30.0	41.0	11.0 / 9.7	W5 (이웃 아님) → S3 · S4
4	W4	42.0	56.0	14.0 / 12.0	없음, W4의 두 번째 crash → S8
5	W2	52.0	62.0	10.0 / 10.0	W4 (이웃 아님)
6	W7	67.0	79.0	12.0 / 15.0	없음. 다음 후보 81초 > 80초 → 더 이상 crash 없음

S 1 · S 2

기동과 정상 흐름 (crash 없음)



S1. 기동과 Ring 구성

그림의 기동 순서대로 진행. 하나라도 HELLO나 RING_OK를 보내지 않으면 START는 나가지 않는다. Launcher는 READY 10개를 받은 뒤에야 스케줄 시작. 로그: RING | SUCCESS | 10/10 RING_OK, START

S2. 정상 흐름

- REQ → ASSIGN → step마다 Buddy에게 CKPT → RESULT → ACK → 다음 REQ
- Queue가 비었을 때만 REQ (pull 방식)
- REQ는 ACK를 기다린 뒤 보내도, RESULT 직후 보내도 된다 (의사코드는 RESULT 직후 방식)
- HB는 계산과 상관없이 1초마다

S 3

단일 **crash** → 감지 → 인계 → 재합류 → 새 **Task (W9)**

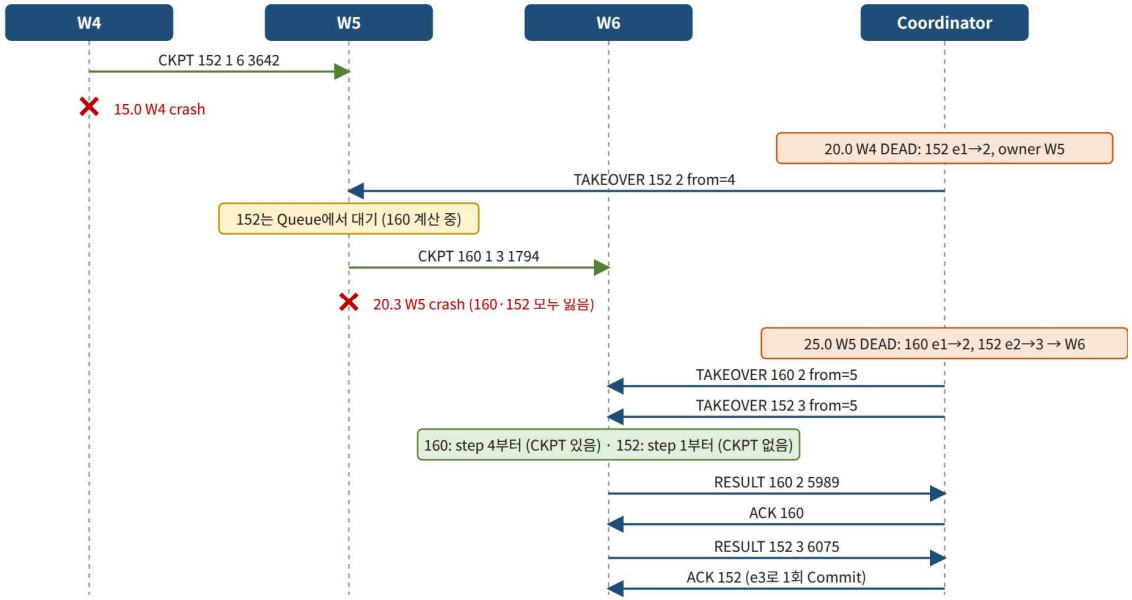


시각 (sec)	W9	Coordinator	W10 (다음 Worker)
29.9 / 29.95	Task 298 step 6 → CKPT 298 1 6 … / 마지막 HB	lastHB[9] = 29.95	Store[298] 갱신
30.0	CRASH → Queue · Store 비움, 침묵 (연결 유지)	모름 (EOF 없음)	—
33.0 / 35.0	침묵	W9 SUSPECT (3.05초) / DEAD → TAKEOVER 298 2 from=9	CKPT 있음 → step 7부터
36.6	—	RESULT 298 2 → ACK (1회 Commit)	RESULT 298 2 …
41.0 → 41.14	RESTART → REJOIN 9 → WELCOME → REQ 9 → ASSIGN 402 1	41.02 DEAD → ALIVE, rejoin_count[9] = 1	—

지표: 감지 지연 = 35.0 − 29.95 = 5.05초 · 복구 시간 = 36.6 − 35.0 = 1.6초 · CKPT 절약 6/10 step · 재합류 지연 = 41.14 − 41.02 = 0.12초

S 4 · S 5

동시에 2개 crash — 이웃이 아닌 경우와 이웃한 경우



S4. 이웃 아님 (W5와 W9, 30 ~ 34초)

다음 Worker가 W6, W10으로 겹치지 않음 → 서로 독립. CKPT도 살아 있는 Buddy에 있음. DEAD 시각이 25초 · 35초로 달라 따로 처리 (같은 검사에서 둘이 DEAD면 번호순)

시각	S5. 이웃 (W4 → W5, 20.3 ~ 27초)
15.0	W4: 152 step 6 → CKPT(3642)를 W5에 → CRASHED
20.0	W4 DEAD → TAKEOVER 152 2 → W5 (방금 160을 시작해 152는 Queue에)
20.3	W5: 160 step 3 CKPT(1794)를 W6에 → CRASHED (160 · 152 모두 잃음)
25.0	W5 DEAD → W6: TAKEOVER 160 2, TAKEOVER 152 3
25.93	160: step 4부터 (CKPT) → RESULT 160 2 5989 → ACK
26.93	152: step 1부터 (CKPT 소실) → RESULT 152 3 6075 → ACK
27.0	W4 rejoin → ASSIGN 271 1 (152를 전혀 모름)

포인트: ① W5가 인계받았던 152까지 "W5 소유"로 기록 → W5 DEAD 때 빠짐없이 다시 인계 ② CKPT가 사라지면 느려질 뿐 결과는 정확 ③ 152의 최종 epoch는 3 — e1 · e2의 늦은 RESULT는 모두 거부

S 6 ~ S 9

다음 Worker가 DEAD · SUSPECT인 경우, 반복 crash, 경계 상황

S6. 다음 Worker가 이미 DEAD (가상)

W5가 먼저 DEAD인 상태에서 W4 crash → W5를 건너뛰고 W6에게. W4의 CKPT는 W5에 있었지만 W5가 버렸으므로 W6은 step 1부터

S7. 다음 Worker가 SUSPECT (가상)

W5가 17.0초에 crash → 20.0초 W4 DEAD 때 W5는 SUSPECT → 건너뛰고 W6에게 (필수 규칙). 넘겼더라도 결과는 정확하지만 느림

S8. 같은 Worker 반복 crash (W4, 42초)

27초에 재합류해 271, 283 …을 처리하다 다시 crash. 절차는 첫 번째와 같음. 56초 REJOIN → rejoin_count[4] = 2 → 또 새 Task

S9. 경계 상황	무슨 일이 일어나나
crash 직전에 보낸 RESULT가 늦게 도착	인계 전에 도착하면 유효한 Commit (그 뒤 FD는 DONE Task를 건너뜀). 인계 후면 REJECT_STALE (보낸 Worker가 아직 DEAD면 DEAD, 아니면 DONE 또는 EPOCH)
crash 직전에 보낸 REQ	DEAD 전에 처리되면 배정 (FD가 DEAD 때 그 Task도 인계). DEAD 후면 무시
RESTART가 DEAD 판정보다 빠름	규칙상(D ≥ 10초) 생기지 않지만, 생기면 RETRY_LATER → 2초 뒤 REJOIN 재시도
rejoin 후 옛 Task의 ACK · REJECT 도착	Worker는 모르는 Task이므로 무시
PENDING이 바닥난 뒤 rejoin	REQ에 NO_TASK. 80초 이후 crash 없음 규칙으로 정식 실행에서는 대개 생기지 않음

S 1 0

종료 (Graceful Termination)

~100.0	PENDING 소진 → REQ에 NO_TASK. Worker는 이후 REQ 없이 블로킹 대기
~101.2	1,000번째 Commit → 그 Dispatcher가 종료 CondVar로 Main을 깨움
~101.3	Main: 합계 · commit_count 검증 → VERIFY 로그 · 요약 → 10개 모두에 BYE (CRASHED인 Worker도 BYE는 처리). 이후 FD는 판정 중지
~101.4	Worker: 요약 로그 후 종료 → Launcher 제어 연결 EOF → Launcher: 10개 EXITED 확인 → 예정 RESTART 취소 → Launcher.txt 마감
~101.6	Main: STOP × 6 → Dispatcher 종료 → 종료 플래그로 Listener(select 타임아웃 또는 Socket close)와 FD(다음 주기) 종료 → 모든 Thread join → TERMINATE. BYE 뒤의 EOF는 정상 종료로 보고 Socket을 닫음

종료 작업(VERIFY, BYE, join)은 반드시 Main Thread가 합니다. 종료처럼 “모두 깨어나야 하는” 경우는 notify_all 또는 STOP 항목을 Dispatcher 수만큼 넣습니다.

PART 06

실행 · 로그 · 평가 · 제출

실행 방법과 출력 예시 · 로그 형식 · 지표와 정합성 · 필수 조건 · 제출물 · 로드맵 · FAQ

HOW TO RUN

실행 방법과 출력 예시 (Python 기준, 형식은 자유)

```
# [원격 호스트] Coordinator
python3 coordinator.py --host 0.0.0.0 --port 9000

# [로컬 PC] Chaos Launcher가 Worker 10개를 실행하고 장애를 주입
python3 launcher.py --coord <공인IP>:9000 --workers 10 --peer-ip 127.0.0.1 \
                    --peer-base 7000 --step-delay 100 --seed 42
# 내부적으로: python3 worker.py --id i --coord <IP>:9000 --peer-ip 127.0.0.1 \
#             --peer-port $((7000+i)) --step-delay 100
```

```
====================== HW#3 RESULT ======================
 Ring    : W1→W2→…→W10→W1 (10/10 RING_OK)
 Elapsed : 101.2 s (START → last COMMIT)  Throughput : 9.88 task/s
 Tasks   : done=1000 commits=1000 double_commit=0
 Sum     : 5761455 (expected 5761455) → PASS
 Crashes : DEAD=6 REJOIN=6 TAKEOVER=7 REJECT_STALE=0 max_concurrent_dead=2
 # worker lastHB DEAD  detect takeover(to)  recover rejoin 1st_task
 1 W4    14.95 20.00 5.05s  152→W5        6.9s*  27.02  271 (0.10s)
 2 W5    19.95 25.00 5.05s  160,152→W6    1.9s   34.03  337 (0.09s)
 3 W9    29.95 35.00 5.05s  298→W10       1.6s   41.02  402 (0.12s)
 4 W4    41.96 47.00 5.04s  405→W5        1.3s   56.04  561 (0.08s)
 5 W2    51.97 57.00 5.03s  523→W3        1.5s   62.03  618 (0.11s)
 6 W7    66.95 72.00 5.05s  690→W8        1.4s   79.02  798 (0.10s)
```

```
[launcher] seed=42 workers=10
[launcher] all READY (10/10) → t=0
[t=15.00] CRASH W4 (D=12.0s)  down=[W4]
[t=20.30] CRASH W5 (D=13.7s)  down=[W4,W5]
[t=27.00] RESTART W4
[t=27.03] REJOINED W4         down=[W5]
[t=30.00] CRASH W9 (D=11.0s)  down=[W5,W9]
 …
[t=79.03] REJOINED W7         down=[]
[t=80.00] no more crashes
```

** #1의 152는 W5 crash로 한 번 더 인계되어 복구가 길어짐(S5). 수치와 Task 번호는 설명용 예시(실제 오름차순 배정과 정확히 맞지 않을 수 있음) — 각 조는 실제 측정값을 Readme 표로*

LOGGING

로그 형식과 예시 (S5 이웃 동시 crash 구간)

형식: [HH:MM:SS.mmm] NODE | EVENT | STATUS | message NODE = COORD, WORKERi, LAUNCHER · STATUS = SUCCESS, FAIL, INFO, WARN

```
[14:30:00.000] COORD | START    | SUCCESS | ring ready (10/10 RING_OK). START sent
[14:30:05.000] COORD | POOL     | INFO    | queue=0 pending=948 assigned=10 done=42 alive=[1..10]
[14:30:18.000] COORD | SUSPECT  | WARN    | W4 no HB for 3.05s
[14:30:20.000] COORD | DEAD     | INFO    | W4 no HB for 5.05s (lastHB=14.95). owned=[152] next_alive=W5 dead_now=[W4]
[14:30:20.001] COORD | TAKEOVER | SUCCESS | task=152 epoch 1->2 owner W4->W5
[14:30:23.000] COORD | SUSPECT  | WARN    | W5 no HB for 3.05s
[14:30:25.000] COORD | DEAD     | INFO    | W5 no HB for 5.05s (lastHB=19.95). owned=[152,160] next_alive=W6 dead_now=[W4,W5]
[14:30:25.001] COORD | TAKEOVER | SUCCESS | task=152 epoch 2->3 owner W5->W6
[14:30:25.001] COORD | TAKEOVER | SUCCESS | task=160 epoch 1->2 owner W5->W6
[14:30:25.930] COORD | COMMIT   | SUCCESS | task=160 e2 by W6 value=5989 commit_count=1
[14:30:26.930] COORD | COMMIT   | SUCCESS | task=152 e3 by W6 value=6075 commit_count=1
[14:30:27.020] COORD | REJOIN   | SUCCESS | W4 DEAD->ALIVE rejoin_count=1. WELCOME
[14:30:27.120] COORD | ASSIGN   | SUCCESS | task=271 e1 -> W4 (first task after rejoin)
[14:31:41.200] COORD | VERIFY   | SUCCESS | done=1000 commits=1000 double=0 sum=5761455 PASS
# Worker6.txt
[14:30:25.010] WORKER6 | TAKEOVER | SUCCESS | task=160 e2 from W5. ckpt=(e1, step 3, 1794)-> resume step 4
[14:30:25.011] WORKER6 | TAKEOVER | SUCCESS | task=152 e3 from W5. no ckpt-> start step 1
# Launcher.txt
[14:30:20.300] LAUNCHER | CRASH   | INFO    | #2 victim=W5 (random from RUNNING=[1,2,3,5,…,10]) D=13.7s
```

읽는 법: DEAD 줄의 owned = 그 순간 owner였던 Task(인계받은 것 포함), next_alive = 인계 대상, dead_now = 그 순간 DEAD 목록(최대 2개). TAKEOVER 줄의 epoch 변화로 같은 Task가 몇 번 인계되었는지 추적

LOGGING

반드시 남겨야 할 로그 이벤트

EVENT	기록하는 노드	언제 / 무엇을
INIT / CONNECT	COORD, WORKER	시작, HELLO 등록
RING / START	COORD, WORKER	RING 통보 · Buddy 연결 · RING_OK, START
ASSIGN / RESULT / COMMIT	COORD, WORKER	Task 배정, 결과 전송, ACK(반영)
CKPT	WORKER	CKPT 송수신 (양이 많으면 10초마다 건수 요약 가능)
HB / SUSPECT / DEAD	COORD	HB 요약, 3초 · 5초 판정 (마지막 HB 시각 포함)
TAKEOVER	COORD, WORKER	task, epoch 변화, from → to, CKPT 사용 여부와 재개 step
CRASH / RESTART	LAUNCHER, WORKER	장애 주입과 그 수행 (Worker는 비운 항목 수 기록)
REJOIN / WELCOME	COORD, WORKER	재합류 요청 · 승인, rejoin_count
REJECT_STALE	COORD, WORKER	옛 결과 거부 (이유)
POOL	COORD	5초마다 Queue 길이, PENDING / ASSIGNED / DONE, ALIVE 목록
VERIFY / TERMINATE	COORD, 모두	검증 결과, 종료

로그로 보여 줘야 할 것

- crash마다 한 주기 전체:
- CRASH → DEAD → TAKEOVER → REJOIN → ASSIGN
- Coordinator.txt · Worker1~10.txt · Launcher.txt가 모두 같은 실행에서 나왔을 것
- 모든 로그 메시지의 명세는 AllDefinedLogs.txt에
- Launcher와 Coordinator 로그는 실제 시각 (HH:MM:SS.mmm)으로 대조

EVALUATION

평가 지표와 정합성 확인

지표	정의 (측정 위치)
처리량 / 실행 시간	[Coord.] 1,000 ÷ (START ~ 마지막 COMMIT)
감지 지연 (crash마다)	[Coord.] 마지막 HB 수신 ~ DEAD 판정 (5~6초 예상)
복구 시간 (crash마다)	[Coord.] DEAD ~ 그 crash로 인계된 Task 모두 Commit (인계할 Task가 없었으면 "—")
재합류 지연	[Coord.] REJOIN 수신 ~ rejoin 후 첫 ASSIGN
rejoin 후 처리 Task 수	[Coord.] 재합류 후 그 Worker가 Commit한 Task 수
CKPT 절약률	인계 Task에서 다시 계산하지 않은 step 비율 = crash별 Σ(재개 step − 1) ÷ (인계 Task 수 × 10), 재개 로그 없으면 0. 예: #2 = 3/20 (Worker 로그로 계산 → Readme)
이중 Commit	commit_count ≥ 2인 Task 수 (반드시 0)

정합성 확인 항목 — 모두 만족 → PASS (방법은 Readme에)

① 1,000 Task DONE, commit_count 합 = 1,000

② 합계 = 5,761,455

③ Launcher의 CRASH 수 = Coordinator의 DEAD 수 = REJOIN 수

④ 어느 순간에도 DEAD 수 ≤ 2 (dead_now 기록)

⑤ crash마다 재합류 후 새 Task 수행 (나눠 줄 Task가 남아 있는 경우)

⑥ Coordinator 로그의 COMMIT 수 = 1,000 (Worker의 ACK 수는 더 적을 수 있음 — Commit 직후 crash하면 ACK가 버려짐)

REQUIREMENTS

필수 구현 조건과 주요 감점 사유

하나라도 미충족 시 과제 전체 0점

- 모든 통신 TCP Socket. Coordinator는 물리적 원격 호스트, Worker 10개와 Launcher는 같은 로컬 PC 한 대. 정식 실행 --step-delay 100, 최종 로그가 이 구성에서 얻어진 것
- Ring 구성(RING_OK 10개) 후에만 계산 시작. Dispatcher 6개 고정 Thread Pool + Request Queue (요청 · 연결마다 Thread 생성 금지), Task별 Mutex
- 가상 crash 동안 TCP 연결 유지, DEAD는 HB 타임아웃으로만 (Launcher ↔ Coordinator 통신 금지), 인계는 다음 ALIVE Worker에게, CKPT가 있으면 다음 step부터
- Launcher가 victim을 무작위로, 임의의 순간 최대 2개 crash. crash마다 DEAD → TAKEOVER → REJOIN → 새 Task 수행 (Task가 남아 있는 경우)
- 1,000 Task DONE · 모든 commit_count = 1 · 합계 5,761,455

△ 주요 감점 사유

- 대기를 Condition Variable 대신 sleep 반복 확인(busy-waiting)으로 구현
- Lock 순서 위반, Lock을 잡은 채 send, REQ의 확인과 배정을 분리
- CKPT가 있는데도 항상 step 1부터 재계산, SUSPECT 단계 없음, 다음 ALIVE 규칙 미준수
- crash 중 HB · 메시지가 새어 나감, crash 때 상태를 지우지 않아 옛 Task를 이어 계산
- 로그에서 crash 주기(CRASH → DEAD → TAKEOVER → REJOIN → ASSIGN)를 추적할 수 없음
- Readme 필수 항목 누락 또는 실제 구현과 다름

SUBMISSION

제출물 — G조이름HW3.zip (조별 1인 제출, 압축 오류 시 0점)

소스 코드

전체 소스 (Coordinator · Worker · Chaos Launcher)

결과 로그 12개

Coordinator.txt · Worker1~10.txt · Launcher.txt — 원격 구성 정식 실행 1회분, 모두 같은 실행

AllDefinedLogs.txt

로그 EVENT를 포함한 모든 로그 메시지 명세 및 설명

download.txt

5분 내외의 시연 동영상(G조이름HW3.mp4) 다운로드 링크 (링크 오류·권한·재생 오류는 감점)

Readme.txt 필수 항목

- 조원 이름 · 학번 · 역할
- 배포 정보: 원격 호스트(서비스 · 인스턴스 · OS), 실행 명령, 시간대, seed (공인 IP는 가려도 됨)
- 구성요소 · 메시지 형식, 컴파일 · 실행 방법 (실행 불가 시 0점)
- 장애 감지(HB · FD) · Checkpoint · Takeover · Epoch · Rejoin 설계, Worker의 crash 처리 방법
- 인계받은 Task와 진행 중 Task의 처리 방법
- crash별 지표 표와 정합성 확인 방법 · 결과
- 추가로 언급할 내용 · 한계 · 추가 구현 사항 (자유)

체크리스트

원격 구성 로그? · RING_OK 10개 후 START · Dispatcher 6개 고정 · CV는 while · Lock 순서 · crash 중 메시지 0건 · 다음 ALIVE 인계 · 동시 crash ≤ 2 · VERIFY PASS · zip이 정상적으로 풀리는가?

ROADMAP

이렇게 개발하면 쉬워요 — 권장 순서

1 **소수 함수**
19쪽 검증 값과 일치 (1,229 / 9,592 / 6,075 …)

2 **기동 + Ring 구성**
로컬 10개로 HELLO → RING → RING_OK → START 성공

3 **정상 계산**
crash 없이 1,000 Task, 합계 정답, commit_count 모두 1

4 **HB + Failure Detector**
Worker 하나의 HB를 끄면 약 5초 뒤 DEAD

5 **CKPT + Takeover**
수동 CRASH 1회로 인계, CKPT 다음 step부터 재개

6 **RESTART + Rejoin**
REJOIN → WELCOME → 새 Task, 같은 Worker 반복 crash

7 **Launcher (무작위, 최대 2)**
무작위 스케줄 · 이웃 동시 crash에서도 정답

8 **원격 배포 + 정식 실행**
필수 조건 충족, 로그 · 지표 정리 → 영상

PITFALLS

흔한 실수 TOP 9

실수	증상	해결
crash 중에도 HB가 한 번씩 나감	DEAD가 안 되거나 늦게 됨	HB Thread가 state를 CondVar로 확인
crash 때 로컬 Queue · Store를 안 비움	rejoin 후 옛 Task 계산 → REJECT_STALE 다수	on_crash에서 모두 비우기
REQ 확인과 배정 분리	가끔 999/1000에서 멈춤	Worker Table Lock 안에서 함께
인계 대상을 고정 Buddy로	이웃 동시 crash 때 죽은 Worker에게 넘김	다음 ALIVE 규칙
DEAD 처리에서 "인계받은 Task"를 빠뜨림	S5에서 152가 영원히 미완료	owner = d인 모든 ASSIGNED 검색
Lock 순서 역전 / Lock 잡은 채 send	Deadlock, 전체 멈춤	WT → Task 순서, Lock 밖에서 send
crash_gen을 기다리기 전에 읽음 / 전송 직전 확인 없음	rejoin 후 Task를 버려 멈춤 / crash 중 CKPT · RESULT가 샘	꺼내는 순간 gen 읽기 + send_if_alive
CondVar를 if로 기다림	가끔 빈 Queue에서 pop 오류	while로 조건 재확인
recv 한 번 = 메시지 하나로 가정	원격에서만 파싱 오류	줄 단위 버퍼링

F A Q

자주 묻는 질문

Q **Coordinator에게 “이 Worker가 crash했다”고 알려 주면 안 되나요?**

안 됩니다. Coordinator는 Launcher와 통신하지 않고 HB가 끊긴 것만 보고 판단해야 합니다. 이것이 과제의 핵심입니다.

Q **crash 중인 Worker에게 TAKEOVER가 가면?**

그 Worker는 받은 메시지를 버립니다. 그 Worker가 DEAD로 판정될 때 다음 ALIVE에게 다시 인계되므로 결과는 정확합니다.

Q **인계받은 Task와 진행 중 Task, 무엇을 먼저?**

자유. 권장은 진행 중 Task를 끝낸 뒤 인계 Task (중간에 바꾸려면 멈춘 Task의 step · partial을 Queue에 다시). Readme에 적기

Q **같은 Worker가 연속으로 뽑혀도 되나요?**

됩니다. 단, 재합류(REJOINED)해서 RUNNING인 Worker만 뽑힐 수 있습니다.

Q **crash 횟수나 동시 2개 crash가 반드시 몇 번?**

의무는 없습니다. 다만 어떤 무작위 스케줄(이웃 동시 crash 포함)에서도 정확해야 합니다. 사용한 seed는 Readme에.

Q **Worker를 여러 PC에 나눠도 되나요? 언어는?**

아니요, Launcher와 같은 PC 한 대 (Peer 포트 7000 + i). 언어 · 라이브러리는 자유, Thread · Mutex · CV를 직접 사용

WRAP-UP

마무리 요약

감지

HB 타임아웃만으로 판단
3초 SUSPECT · 5초 DEAD
EOF 없는 가상 crash

인계

다음 ALIVE Worker에게
CKPT가 있으면 다음 step부터
인계받은 Task도 빠짐없이

정확히 한 번

Epoch + owner를 Task Lock 안에서 비교
옛 결과는 REJECT_STALE
이중 Commit 0건

재합류

RESTART → REJOIN → WELCOME
이전 기억 없이 새 Task
몇 번이든 반복

목표

crash가 몇 번 나도 1,000 Task 모두 DONE · commit_count 모두 1 · 합계 5,761,455 → VERIFY PASS

DUE: 2026.10.26 (월) 23:59 · Assigned: 2026.10.13 (화) 10:30 · 제출: G조이름HW3.zip (조별 1인)